

游戏引擎面试指南

100 道高频真题与系统解法

C++ · 3D 数学 · 实时渲染 · 引擎架构 · GPU API · 性能优化

定位：面向游戏引擎 / 图形渲染 / Engine Programmer / Graphics Programmer 的系统化面试复习书。

组织：题目 → 30 秒回答 → 深入拆解 → 工程 Gotcha → 追问树 → 面试官视角。

版本：2026.09 · 非官方题库 · 公开面经与岗位能力模型交叉整理。

使用说明与资料口径

本书参考“题目驱动、逐层追问、强调思路与易错点”的经典技术面试书组织方式，但不复制任何受版权保护书籍的原文、例题解答或版式。

“真题”口径：公开候选人面经只代表个人回忆，并非公司官方题库。本书对可核验的面试主题进行**改写与归纳**，用 [M1]~[M6] 标识；其余题目标为“高频能力重构”，来源于多个岗位共同要求与引擎工程知识图谱。

建议使用方式：

1. 第一次：只看每题“30 秒回答”，建立知识地图。
2. 第二次：遮住答案，口述 2~5 分钟，再对照“深入拆解”。
3. 第三次：只练“高频追问树”，把单点知识练成可延展的面试对话。
4. 项目复盘：每个知识点都强制回答“我在哪段代码里真正实现/调试过它？”

引擎面试的能力地图

能力域	面试真正关心的东西	优先级
C++ / Memory	对象模型、ownership、allocator、cache、data layout	
并发 / OS	Job System、memory model、虚拟内存、线程所有权	
3D 数学	坐标变换、法线、Quaternion、求交、插值	
实时渲染	Raster、Depth、Forward/Deferred、PBR、Shadow	
引擎架构	Asset、Serialization、RHI、Render Graph、Streaming	
动画/物理/网络	实时模拟、约束求解、确定性、延迟隐藏	
GPU / Profiling	Vulkan 同步、Bindless、GPU-driven、性能定位	

推荐答题模板：从八股到引擎工程

建议把每道题组织成下面五层：

1. **定义**：它是什么？一句话完成。
2. **机制**：底层为什么这样工作？数据怎样流动？
3. **Trade-off**：它解决什么，又付出什么？
4. **Engine Context**：在 renderer / asset / job / physics 中如何落地？
5. **Profiling / Failure**：什么时候会错，怎么证明、怎么定位？

真正拉开差距的通常是第 4、5 层。

目录

使用说明与资料口径	ii
引擎面试的能力地图	iii
推荐答题模板	iv
第一章 C++ 对象模型与语言机制	1
Q01 C++ 虚函数和运行时多态到底是怎么实现的?	2
Q02 为什么基类析构函数经常必须是 virtual?	3
Q03 为什么构造函数不能是 virtual? 构造期间调用 virtual function 会怎样?	4
Q04 unique_ptr、shared_ptr、weak_ptr 底层有什么区别?	5
Q05 shared_ptr 为什么会发生循环引用? 如何从设计上避免?	6
Q06 左值、右值、std::move、移动构造究竟解决什么问题?	7
Q07 std::vector 扩容时发生了什么? 哪些引用会失效?	8
Q08 vector 为什么常常比 list 更快, 即使 list 插入是 O(1)?	9
Q09 new/delete 与 malloc/free 有什么区别? 为什么不能混用?	10
Q10 C++ Reflection 为什么难? 游戏引擎如何实现反射?	11
第二章 内存、STL 与数据导向设计	12
Q11 游戏引擎为什么特别在意内存对齐?	13
Q12 Stack 和 Heap 的区别是什么? 实时引擎如何选择?	14
Q13 如何设计一个固定大小 Memory Pool?	15
Q14 Arena / Linear Allocator 为什么适合每帧临时对象?	16
Q15 AoS 和 SoA 有什么区别? 为什么 SoA 常用于热点系统?	17
Q16 ECS 为什么越来越常见? 它解决了什么, 又带来什么成本?	18
Q17 Hash Map 和红黑树怎么选?	19
Q18 Object Pool 适合什么场景? 如何避免把它用成“内存垃圾场”?	20
Q19 什么是 Cache Miss? 为什么游戏引擎比普通业务代码更在意?	21
Q20 什么是 False Sharing? 如何定位与修复?	22

第三章 操作系统、多线程与并发	23
Q21 Process 与 Thread 的区别是什么?	24
Q22 Context Switch 为什么贵?	25
Q23 什么是 Job System? 为什么比“一个子系统一个线程”更可扩展?	26
Q24 Mutex、Spinlock、Semaphore 怎么选?	27
Q25 什么是 Deadlock? 四个必要条件是什么?	28
Q26 如何实现一个 SPSC 无锁环形队列?	29
Q27 memory_order_relaxed / acquire / release 是什么?	30
Q28 虚拟地址如何转换成物理地址? Page Fault 发生了什么?	31
Q29 Game Thread 和 Render Thread 为什么要分开?	32
Q30 Triple Buffering 与 Frames in Flight 有什么作用?	33
第四章 数据结构与算法	34
Q31 10 万个数找最大的 1 万个, 怎么做?	35
Q32 实现 A*, 为什么它比 Dijkstra 更适合游戏寻路?	36
Q33 A* 在开放世界地图里为什么仍可能很慢?	37
Q34 Bresenham 画线算法为什么能避免浮点数?	38
Q35 二叉树最近公共祖先怎么做?	39
Q36 最长合法括号如何做到 O(N)?	40
Q37 所有节点出度 1, 如何寻找最长环?	41
Q38 如何设计空间哈希解决大量对象邻域查询?	42
Q39 Quadtree、Octree、BVH 有什么区别?	43
Q40 动态对象很多时 BVH 怎么更新?	44
第五章 3D 数学与几何	45
Q41 游戏渲染中有哪些坐标空间?	46
Q42 齐次坐标为什么需要第四维?	47
Q43 为什么 Normal 不能直接乘 Model Matrix?	48
Q44 Quaternion 为什么适合表示旋转?	49
Q45 SLERP 和 LERP 有什么区别?	50
Q46 如何判断一个点/包围体是否在视锥体中?	51
Q47 Ray 与 Triangle 如何求交?	52
Q48 AABB 和 OBB 的区别?	53
Q49 什么是重心坐标?	54
Q50 为什么纹理属性必须做透视矫正插值?	55
第六章 实时渲染管线	56

Q51 请完整讲一遍现代 GPU Rasterization Pipeline。	57
Q52 Rasterization 本质上在做什么?	58
Q53 Vertex Shader 与 Fragment Shader 谁调用次数更多?	59
Q54 什么是 Early-Z? 它为什么能加速?	60
Q55 Early-Z 什么时候可能失效或受限?	61
Q56 Hi-Z / Hierarchical Z 是什么?	62
Q57 Forward 和 Deferred Rendering 如何比较?	63
Q58 Forward+ / Clustered Rendering 为什么出现?	64
Q59 为什么 Deferred GBuffer 可以不保存 World Position?	65
Q60 Instancing 为什么能减少 Draw Call?	66
第七章 PBR、阴影、GI 与 Ray Tracing	67
Q61 写出 Rendering Equation, 并解释每一项。	68
Q62 Cook-Torrance BRDF 怎么理解?	69
Q63 Metallic/Roughness PBR Workflow 是什么意思?	70
Q64 Normal Map 为什么通常存在 Tangent Space?	71
Q65 Shadow Mapping 原理是什么?	72
Q66 Shadow Acne 和 Peter Panning 为什么出现?	73
Q67 PCF 和 PCSS 有什么区别?	74
Q68 Path Tracing 为什么会有噪声? 收敛速度如何理解?	75
Q69 什么是 Importance Sampling?	76
Q70 BVH 在 Ray Tracing 中解决什么问题?	77
第八章 引擎架构与资源系统	78
Q71 如果从零设计游戏引擎, 最核心模块有哪些?	79
Q72 Scene Graph 和 ECS 有什么关系?	80
Q73 游戏引擎 Serialization 系统如何设计?	81
Q74 Asset Pipeline 为什么不能直接运行时读取 FBX/PSD?	82
Q75 如何设计 Asset GUID?	83
Q76 什么是 Resource Streaming?	84
Q77 Game Thread、Render Thread、RHI Thread 如何交互?	85
Q78 什么是 Render Graph?	86
Q79 为什么 Shader Variant 会爆炸? 如何控制?	87
Q80 如何设计跨平台 RHI?	88
第九章 动画、物理、AI 与网络	89
Q81 Skeletal Animation 如何工作?	90

Q82 为什么 Linear Blend Skinning 会出现 Candy Wrapper?	91
Q83 FK 与 IK 有什么区别?	92
Q84 Animation State Machine 与 Blend Tree 分别解决什么?	93
Q85 Physics Fixed Timestep 为什么重要?	94
Q86 Collision Detection 为什么分 Broad Phase 和 Narrow Phase?	95
Q87 GJK 在解决什么问题?	96
Q88 NavMesh 为什么比规则 Grid 更适合 3D 游戏角色?	97
Q89 Client Prediction 为什么需要 Server Reconciliation?	98
Q90 Lockstep 和 State Replication 如何选择?	99
第十章 GPU API、现代引擎与性能优化	100
Q91 Vulkan 中 Fence、Semaphore、Barrier 分别解决什么?	101
Q92 为什么 Pipeline Barrier 写得太保守会掉性能?	102
Q93 什么是 Bindless Rendering?	103
Q94 GPU-Driven Rendering 是什么?	104
Q95 Compute Shader 可以在游戏引擎里做什么?	105
Q96 一帧只有 20 FPS, 你如何定位问题?	106
Q97 为什么移动 GPU 和桌面 GPU 优化思路不同?	107
Q98 Nanite 到底解决了什么问题?	108
Q99 Lumen 的设计目标是什么?	109
Q100 系统设计: 如何设计一个稳定 60 FPS 的开放世界引擎?	110
附录 A 30 题冲刺清单	111
附录 B 项目追问模板	112
附录 C 资料来源与进一步阅读	113
结语: 从“知道”到“能定位”	115

第 1 章

C++ 对象模型与语言机制

本章目标

从“会写 C++”升级到理解 ABI、对象生命周期、资源所有权和引擎级元编程。

必须掌握：虚表与对象生命周期；智能指针与 ownership；move/noexcept；vector 失效规则；反射/codegen

Q01 · C++ 虚函数和运行时多态到底是怎么实现的？

难度： · 公开面经/官方资料可核验：[M2], [M5]

30 秒回答

主流 C++ ABI 通常为多态对象保存 `vptr`，`vptr` 指向 `vtable`；虚调用通过对象的动态类型选择最终函数。但标准并不规定必须使用虚表。

深入拆解

面试中不能只说“有一张虚函数表”。真正要讲清楚对象布局、动态分派成本与生命周期。单继承常见一个 `vptr`；多继承可能需要多个子对象视角下的 `vptr` 与 `this` 调整；虚继承还会引入额外的基类定位信息。虚调用的直接成本通常只是一次间接加载与间接跳转，更大的成本往往来自阻碍内联、分支预测和数据布局。现代编译器在动态类型可证明时可以 `devirtualize`。

核心抓手

- 区分静态类型与动态类型
- 说明 `vptr/vtable` 是主流实现而非标准要求
- 理解多继承下 `this adjustment`
- 知道 `devirtualization` 与 `LTO/PGO` 的关系

工程 Gotcha

把“虚函数一定慢很多”当成结论。真实瓶颈常在缓存和无法内联，而不是多一条指令。

高频追问树

- 多继承对象里有几个 `vptr`？
- 纯虚函数在 `vtable` 中是什么状态？
- `final` 为什么可能帮助优化？
- 构造/析构期间虚调用如何分派？

面试官在听什么

是否真正理解 C++ 对象模型，而不是背诵“虚函数表”四个字。

Q02 · 为什么基类析构函数经常必须是 virtual?

难度： · 高频能力重构

30 秒回答

如果对象会经由基类指针被删除，基类析构函数必须是 virtual，才能沿动态类型正确执行完整析构链。

深入拆解

关键不是“所有基类都加 virtual destructor”，而是 ownership API 的契约。如果一个类型是 polymorphic base 且允许 delete through base pointer，应提供 virtual destructor；若明确禁止这种删除，可以使用 protected non-virtual destructor。虚析构会让类进入多态对象模型并产生相应 ABI/尺寸成本，所以值类型、数学类型、final 类型不应机械添加。

核心抓手

- 理解 delete-expression 的动态析构语义
- 区分接口基类与值类型
- 知道 protected non-virtual destructor 的设计用途

工程 Gotcha

回答成“基类析构必须 virtual”。只有需要经由基类指针销毁时才是必须。

高频追问树

- 析构函数可以是 pure virtual 吗？
- 为什么 pure virtual destructor 仍需要定义？
- `unique_ptr<Base>` 删除 Derived 有什么要求？

面试官在听什么

能否把语言规则与所有权设计联系起来。

Q03 · 为什么构造函数不能是 virtual? 构造期间调用 virtual function 会怎样?

难度: · 公开面经/官方资料可核验: [M2]

30 秒回答

构造函数负责建立对象本身，动态分派所依赖的“完整派生对象”此时尚未形成；构造/析构期间的虚调用只分派到当前正在构造/析构的层级。

深入拆解

构造顺序是 base subobject 先于 derived subobject。基类构造运行时，派生类字段还未初始化，因此如果允许分派到派生实现，派生实现可能读取不存在的状态。主流实现会在不同构造阶段更新对象的虚表指针，使动态类型随生命周期阶段变化。析构时过程相反：派生部分先被销毁，再退化到基类。

核心抓手

- 掌握对象生命周期阶段
- 知道构造/析构内调用虚函数的实际分派
- 理解 factory pattern 为什么可替代“virtual constructor”

工程 Gotcha

把“构造函数不能 virtual”仅解释成“语法不允许”，没有说明对象尚未完整建立的根本原因。

高频追问树

- 如何实现 virtual constructor 语义?
- clone() 应如何设计?
- 构造期间调用纯虚函数会怎样?

面试官在听什么

是否能从对象生命周期解释语言设计。

Q04 • `unique_ptr`、`shared_ptr`、`weak_ptr` 底层有什么区别？

难度： • 公开面经/官方资料可核验：[M2], [M3], [M5]

30 秒回答

`unique_ptr` 表示独占所有权，通常只保存裸指针和 deleter；`shared_ptr` 通过 control block 维护 strong/weak 引用计数；`weak_ptr` 不增加 strong count，用于观察共享对象而不延长生命周期。

深入拆解

`shared_ptr` 的成本包括控制块分配、引用计数原子操作和更复杂的对象生命周期。`make_shared` 往往把对象与控制块合并分配，减少一次 allocation 并改善 locality，但会改变对象内存何时真正释放：即使 strong count 为零，只要 weak count 仍存在，合并块可能继续驻留。游戏引擎中应优先建立清晰 ownership，而不是把 `shared_ptr` 当“安全裸指针”。

核心抓手

- 控制块通常包含 strong/weak count、deleter/allocator
- 理解 `make_shared` 的单次分配
- `weak_ptr::lock` 的竞态安全语义
- 知道 `shared_ptr` 别名构造的存在

工程 Gotcha

把 `shared_ptr` 当成“线程安全对象”。引用计数操作线程安全，不代表被指对象本身线程安全。

高频追问树

- `make_shared` 有什么优缺点？
- `shared_ptr` 的两个指针分别是什么？
- `enable_shared_from_this` 解决什么问题？
- 原子引用计数会带来哪些 cache-line 成本？

面试官在听什么

是否理解所有权、控制块与并发成本。

Q05 • shared_ptr 为什么会发生循环引用？如何从设计上避免？

难度： · 公开面经/官方资料可核验：[M3]

30 秒回答

如果两个对象互相持有 shared_ptr，双方 strong count 都无法归零；应把至少一条“非所有权关系”改成 weak_ptr。

深入拆解

循环引用本质上是 ownership graph 中出现强引用环。正确做法不是事后搜索环，而是在架构上定义“谁拥有谁”：树形层级通常由父节点拥有子节点，子节点只观察父节点；事件系统、缓存和 observer 关系也常是弱引用。引擎里 entity/component、asset dependency、editor object graph 都需要明确区分 ownership 与 reference。

核心抓手

- 将对象关系建模为 ownership graph
- 强引用表示生命周期责任，弱引用表示可失效观察
- 避免在全局 manager 与对象间建立双向 shared ownership

工程 Gotcha

仅回答“用 weak_ptr 就行”，没有说明哪一边应该弱、为什么。

高频追问树

- weak_ptr::expired 与 lock 有什么区别？
- 资源缓存应该持强引用还是弱引用？
- GC 引擎为什么可能不使用 shared_ptr？

面试官在听什么

能否先设计生命周期，再选择智能指针。

Q06 · 左值、右值、std::move、移动构造究竟解决什么问题？

难度： · 高频能力重构

30 秒回答

移动语义允许对象把内部资源“转交”给新对象，避免深拷贝；std::move 本身只是类型转换，不执行移动。

深入拆解

对拥有大 buffer、GPU staging memory、动态数组等资源的类型，拷贝可能是 $O(n)$ ，而移动通常只是交换几个指针与长度。实现 move constructor 时要把源对象置于“有效但未指定”的可析构状态。若 move 操作标记 noexcept，std::vector 扩容时更愿意移动而不是为了强异常保证退回到 copy。

核心抓手

- std::move 是 cast，不是操作
- 移动后源对象必须可析构/可赋值
- noexcept 会影响标准容器迁移策略
- 理解 perfect forwarding 与 move 的差别

工程 Gotcha

对 const 对象 std::move 后期待调用 $T(T\&\&)$ 。const T&& 通常不能绑定到需要修改源对象的 move ctor，最终可能仍调用 copy。

高频追问树

- 为什么 move ctor 常写 noexcept？
- return value optimization 与 std::move 的关系？
- 什么时候在 return 上写 std::move 反而妨碍优化？

面试官在听什么

是否理解 move 的资源语义，而不是语法口诀。

Q07 • std::vector 扩容时发生了什么？哪些引用会失效？

难度： • 公开面经/官方资料可核验：[M2]

30 秒回答

容量不足时 vector 申请更大的连续区域，把旧元素 move/copy 到新区域，销毁旧元素并释放旧存储；发生 reallocation 时所有指针、引用和 iterator 都会失效。

深入拆解

vector 的优势是连续内存；代价是容量增长时必须整体迁移。增长倍率由实现决定，不应假设严格 2 倍。reserve 可以提前规划容量，减少分配和迁移；但 reserve 过大也会增加常驻内存。对于非平凡类型，移动是否 noexcept 会影响迁移选择。引擎代码还要警惕把 vector 元素地址长期注册到其他系统。

核心抓手

- size 与 capacity 的区别
- reserve 不改变 size
- reallocation 的 iterator invalidation
- emplace_back 也可能触发整体迁移

工程 Gotcha

认为“尾部 push_back 永远 $O(1)$ ”。准确说是 amortized $O(1)$ ，单次扩容可能 $O(n)$ 。

高频追问树

- vector 插入中间位置发生什么？
- erase 后哪些 iterator 失效？
- small-vector 优化适合什么场景？

面试官在听什么

是否能把容器语义与对象生命周期、缓存和引擎指针稳定性联系起来。

Q08 • vector 为什么常常比 list 更快，即使 list 插入是 $O(1)$?

难度： • 公开面经/官方资料可核验：[M2]

30 秒回答

因为现代 CPU 的瓶颈往往是内存层级而不是算术复杂度。vector 连续存储，cache locality、hardware prefetch、批量 SIMD 都更友好；list 的 pointer chasing 经常产生 cache miss。

深入拆解

Big-O 忽略常数、内存访问和分支行为。list 的 $O(1)$ 插入还要求“你已经有 iterator 指向目标位置”；若需要先查找位置，整体仍可能 $O(n)$ 。每个 list 节点通常还包含两个指针与独立 allocation，增加内存和 allocator 压力。游戏引擎热点路径里，数据连续性通常比理论插入复杂度更重要。

核心抓手

- 连续访问比 pointer chasing 更适合 CPU
- 节点容器有额外指针和分配开销
- 复杂度必须和访问模式一起分析

工程 Gotcha

只比较 $O(n)$ 与 $O(1)$ 就下结论。

高频追问树

- deque 为什么与 vector 性能不同？
- 什么时候 list 真正有优势？
- flat_map 为什么常比 tree map 快？

面试官在听什么

是否具备 data-oriented performance intuition。

Q09 • new/delete 与 malloc/free 有什么区别？为什么不能混用？

难度： · 公开面经/官方资料可核验：[M2]

30 秒回答

malloc/free 只管理原始字节；new/delete 同时涉及对象构造/析构以及匹配的分配函数。通过 new 创建对象后使用 free，会绕过析构且分配/释放协议不匹配，属于未定义行为。

深入拆解

C++ 的 new-expression 概念上包含“调用 operator new 获取内存 + 在该内存上构造对象”；delete-expression 先执行析构，再调用匹配的 operator delete。placement new 则只执行构造，不分配存储。引擎自定义 allocator 常重载 operator new 或完全绕过 new-expression，但仍必须维护对象生命周期与对齐。

核心抓手

- 区分 allocation 与 object lifetime
- 了解 placement new
- new[] 必须匹配 delete[]
- 自定义 allocator 仍需显式构造/析构

工程 Gotcha

把原因仅说成“会内存泄漏”。真正问题是未定义行为，包括析构未执行和 allocator metadata 被破坏。

高频追问树

- placement new 后怎么析构？
- operator new 与 new-expression 有何区别？
- 为什么数组 delete 需要知道元素数量？

面试官在听什么

能否区分“内存”和“对象”两个概念。

Q10 • C++ Reflection 为什么难？游戏引擎如何实现反射？

难度： · 公开面经/官方资料可核验：[M1]

30 秒回答

传统 C++ 缺少完整运行时反射，因此引擎常通过宏 + code generation、Clang AST 或自定义编译工具生成类型元数据。反射支撑序列化、编辑器属性、脚本绑定、GC、RPC 和资产系统。

深入拆解

一个成熟反射系统至少需要稳定 Type ID、Field ID、类型继承关系、属性标签、构造/析构入口和版本迁移策略。仅靠 RTTI/typeid 远远不够。Unreal 的 UHT 属于典型“源代码扫描 + 生成代码”路线；其他引擎也可能用 Clang tooling 解析 AST。难点通常不是“获取字段名”，而是保持编译速度、跨模块 ABI、热重载、版本兼容和编辑器可用性。

核心抓手

- 反射是元数据系统，不只是 typedef
- codegen 能把运行时成本前移到构建阶段
- 反射与 serialization/GC/editor 强耦合
- 稳定 ID 比字段字符串更重要

工程 Gotcha

认为 RTTI 就等于引擎反射。RTTI 通常只提供有限动态类型信息。

高频追问树

- 如何为字段改名做兼容？
- 反射生成代码如何参与增量编译？
- 属性系统如何支持容器/泛型？
- 如何设计 TypeRegistry？

面试官在听什么

是否有“工具链 + runtime”一体化思维。

第 2 章

内存、STL 与数据导向设计

本章目标

游戏引擎真正的性能基础不是 Big-O，而是内存布局、缓存、分配策略和访问模式。

必须掌握： alignment；arena/pool；AoS/SoA；ECS；cache miss；false sharing

Q11 · 游戏引擎为什么特别在意内存对齐？

难度： · 公开面经/官方资料可核验：[M2], [M3]

30 秒回答

对齐既影响对象大小，也影响 CPU SIMD、cache line 和 GPU uniform/storage buffer 的合法访问。错误布局会增加 padding、带宽浪费甚至触发平台约束。

深入拆解

CPU 按自然对齐访问通常更高效，某些 SIMD 指令或 GPU ABI 还存在明确 alignment 要求。结构体字段顺序会影响 padding；但“把小字段全放前面”并非普遍最优，应按 alignment 从大到小或结合访问热度设计。跨 CPU/GPU 共享结构体时还必须遵守 std140/std430/HLSL packing 等规则，不能只看 sizeof(C++ struct)。

核心抓手

- 理解 alignment/padding
- 区分结构体大小优化与访问局部性
- 知道 cache-line alignment 可用于避免 false sharing
- GPU buffer layout 需按 API/shader ABI 约束

工程 Gotcha

为了省几个字节破坏热点字段的局部性，或者认为 C++ sizeof 与 shader 端布局必然一致。

高频追问树

- alignas 有什么作用？
- SIMD 数据为什么常做 16/32 字节对齐？
- std140 和 std430 有何不同？

面试官在听什么

是否能跨 CPU 与 GPU 两端思考数据布局。

Q12 • Stack 和 Heap 的区别是什么？实时引擎如何选择？

难度： · 高频能力重构

30 秒回答

栈通常是线程私有、线性增长、自动生命周期，分配极快；堆支持任意生命周期，但通用 allocator 需要元数据、同步和碎片管理。实时循环应避免不可预测的高频堆分配。

深入拆解

“栈快、堆慢”只是现象。栈分配通常只移动 stack pointer，而通用 heap 要寻找合适块、维护 free list/bin，并可能触发锁。游戏引擎会把不同生命周期映射到不同 allocator：frame scratch 用 linear arena；大量同尺寸对象用 pool；长期资源用 general heap；GPU 内存另有 suballocation。关键是让释放策略与生命周期天然一致。

核心抓手

- 每个线程有独立栈
- heap fragmentation 与 allocator contention
- 按生命周期选择 allocator
- 大对象不一定适合直接上栈

工程 Gotcha

把“局部变量都在栈上”当绝对规律。编译器优化和对象逃逸会改变实际存储。

高频追问树

- 栈溢出为什么常见于递归？
- 为什么大型临时数组适合 frame arena？
- 线程栈大小如何影响 job system？

面试官在听什么

是否能从生命周期和可预测性解释 allocator 选择。

Q13 · 如何设计一个固定大小 Memory Pool?

难度： · 高频能力重构

30 秒回答

预分配一大块内存, 把它切成等大小 block, 用 free list 管理空闲块; allocate/free 都可接近 $O(1)$, 且避免通用堆碎片。

深入拆解

实现时要处理 block alignment、对象构造/析构、double free 检测、调试填充值、线程安全与扩容策略。free list 可以把 next 指针直接存放在空闲 block 本身, 从而没有额外节点分配。若跨线程频繁 allocate/free, 一个全局 lock pool 会成为瓶颈, 可改为 per-thread cache + central pool。

核心抓手

- 预分配 slab/block
- 空闲块内嵌 next 指针
- 分配器与构造器分离
- 并发场景考虑 thread-local cache

工程 Gotcha

只实现 “vector<bool> 标记空闲”, 虽然能工作, 但扫描寻找 free block 会失去 $O(1)$ 特性。

高频追问树

- 如何处理不同对象尺寸?
- 如何检测 use-after-free?
- pool 如何支持增长?
- 如何做 lock-free free list, ABA 怎么办?

面试官在听什么

是否能把算法、内存布局和调试可维护性一起考虑。

Q14 • Arena / Linear Allocator 为什么适合每帧临时对象？

难度： • 高频能力重构

30 秒回答

它只维护一个 offset，分配就是对齐后递增；帧结束统一 reset，不逐对象 free，因此开销小且碎片几乎为零。

深入拆解

Linear allocator 的本质是把“成百上千次释放”替换成一次阶段性回收。它最适合生命周期天然同批结束的数据：render command build、临时可见列表、动画 scratch、查询结果。若对象拥有需要析构的资源，应另外记录 destructor list，或者只放 trivial/arena-owned 数据。常见增强是双缓冲/多帧 arena，保证 GPU 或其他线程仍在读取旧帧数据时不会被 reset。

核心抓手

- bump pointer allocation
- 统一 reset
- 配合 frame-in-flight 设计多套 arena
- 非平凡析构需要额外管理

工程 Gotcha

把长生命周期对象放入 frame arena，导致下一帧 reset 后悬空。

高频追问树

- 如何支持对齐？
- arena 用完怎么办？
- 如何与 Render Thread 的多帧延迟配合？

面试官在听什么

是否把 allocator 与系统生命周期真正绑定。

Q15 • AoS 和 SoA 有什么区别？为什么 SoA 常用于热点系统？

难度： • 高频能力重构

30 秒回答

AoS 把一个实体的全部字段放一起；SoA 把同一字段连续存放。若系统每次只访问少数几个字段，SoA 能减少无关 cache line 读取并更利于 SIMD。

深入拆解

例如粒子更新只需要 position、velocity、life，AoS 中 material、color、debug flags 也可能被一同拉入 cache。SoA 让热点列连续，适合批量计算和向量化。工程上还有 AoSoA：把数据按 SIMD 宽度分块，兼顾局部性与编程便利。最终选择应由“系统访问模式”决定，而不是追求某种教条布局。

核心抓手

- 访问模式决定布局
- SoA 有利于 vectorization
- AoS 对一次处理完整对象更直观
- AoSoA 是常见折中

工程 Gotcha

盲目把所有类型改成 SoA，导致 API 复杂、冷数据管理困难。

高频追问树

- 如何把 Transform 分成 hot/cold data？
- AoSoA 为什么适合 SIMD？
- GPU structured buffer 更偏好哪种布局？

面试官在听什么

是否具备以访问模式驱动数据设计的习惯。

Q16 · ECS 为什么越来越常见？它解决了什么，又带来什么成本？

难度： · 高频能力重构

30 秒回答

ECS 把 identity、data、behavior 分离：Entity 是 ID，Component 是数据，System 批量处理匹配组件。它有利于 dense storage、批处理、多线程和按 archetype 调度。

深入拆解

ECS 的真正收益不是“组合优于继承”这句口号，而是让系统能按组件集合组织紧凑存储。例如 Position+Velocity archetype 可连续遍历，天然适配 Job System。成本包括对象关系表达更间接、编辑器和调试体验更复杂、结构变化可能触发 archetype migration、跨组件随机访问仍会破坏局部性。因此成熟引擎常把 ECS 用在高实体量热点模拟，而不是所有工具/编辑器对象都强行 ECS 化。

核心抓手

- Entity=identity, Component=data, System=behavior
- archetype/chunk 布局决定性能
- 结构变化有迁移成本
- ECS 与 Scene Graph 可以共存

工程 Gotcha

把 ECS 解释成“没有 OOP”。实际很多引擎仍大量使用类，只是热点 runtime 数据采用 ECS。

高频追问树

- Sparse Set 与 Archetype ECS 如何比较？
- 组件增删为什么可能昂贵？
- System 如何建立依赖图？

面试官在听什么

是否理解 ECS 的数据布局本质，而非设计模式标签。

Q17 • Hash Map 和红黑树怎么选？

难度： · 公开面经/官方资料可核验：[M5]

30 秒回答

哈希表平均 $O(1)$ 查找，树结构 $O(\log N)$ 且有序；真实选择还取决于遍历顺序、内存、rehash、确定性和 cache locality。

深入拆解

`std::map` 每个节点独立分配且 pointer-heavy；`unordered_map` 也可能有 bucket/node 开销。对只读或批量构建数据，sorted vector + binary search (flat map) 常具有更好的 cache 性能。网络锁步或 replay 系统还要警惕 `unordered_map` 的迭代顺序不稳定，不能让非确定顺序参与状态哈希或逻辑。

核心抓手

- 复杂度只是第一层
- ordered map 支持范围查询
- hash table 需考虑 load factor/rehash
- 确定性系统要避免不稳定迭代

工程 Gotcha

只回答“要排序用 map，否则 `unordered_map`”。

高频追问树

- 如何防止 hash collision 攻击/退化？
- `flat_map` 适合什么数据规模？
- 自定义 allocator 对 map 有什么意义？

面试官在听什么

是否能从性能与确定性共同做容器选择。

Q18 • Object Pool 适合什么场景？如何避免把它用成“内存垃圾场”？

难度： • 高频能力重构

30 秒回答

适合高频创建/销毁、对象尺寸相近且峰值可估计的对象，如粒子、弹道、临时音源。Pool 通过复用对象减少 allocation 与初始化成本。

深入拆解

对象池的难点是 reset protocol：从 active 退回 inactive 时必须清理所有逻辑状态、事件订阅和外部引用，否则下次复用会携带脏状态。池容量应基于峰值 telemetry，而不是无限增长；可设置 hard cap、fallback allocator 或回收策略。对于非常昂贵 GPU resource, pool 还要与 fence/frame-in-flight 结合，确保资源不再被 GPU 使用。

核心抓手

- 复用对象不等于跳过 reset
- 容量应由实际峰值驱动
- GPU 对象回收必须考虑异步完成
- pool 可减少 allocator 与构造成本

工程 Gotcha

认为 pool 一定省内存。过度预留或永不 shrink 反而会占用大量常驻内存。

高频追问树

- 如何实现 generational handle 防止旧引用误用复用对象？
- pool 和 arena 的生命周期差异是什么？

面试官在听什么

是否理解“复用”带来的状态与生命周期风险。

Q19 · 什么是 Cache Miss? 为什么游戏引擎比普通业务代码更在意?

难度: · 公开面经/官方资料可核验: [M3]

30 秒回答

CPU 运算速度远快于 DRAM; 热点循环如果频繁 L1/L2/L3 miss, 核心会因等待数据而空转。游戏引擎每帧重复处理大批实体, 因此数据布局对总帧时极敏感。

深入拆解

性能分析要区分 instruction cache、data cache、TLB miss、branch misprediction。pointer-heavy scene graph、虚函数对象数组、随机哈希访问都可能造成 memory latency。优化手段包括压缩结构体、hot/cold split、连续数组、prefetch、批处理、减少 indirection。重要的是用 profiler/hardware counter 证明瓶颈, 而不是凭感觉“改成数组”。

核心抓手

- 理解 cache line 与 spatial locality
- 区分 latency-bound 与 compute-bound
- TLB 也是地址翻译缓存
- 用硬件计数器验证

工程 Gotcha

仅把 cache miss 解释为“数据不在缓存里”, 没有说明为什么会改变算法设计。

高频追问树

- AoS/SoA 如何影响 cache?
- 为什么 linked list 典型慢?
- prefetch 何时有效、何时无效?

面试官在听什么

是否有现代 CPU 性能模型。

Q20 · 什么是 False Sharing? 如何定位与修复?

难度: · 高频能力重构

30 秒回答

两个线程写不同变量，但变量落在同一 cache line，缓存一致性协议仍会让该 line 在核心间反复失效，产生“看起来没有共享变量却很慢”的现象。

深入拆解

典型场景是 per-thread counter 紧挨存放。修复可以使用 cache-line padding/alignment、按线程分块、减少共享写、批量累加后再汇总。需要注意 padding 不是越多越好：会增大工作集，可能导致更多 cache/TLB miss。最好借助 profiler 或 hardware performance counter 观察 cache line bouncing。

核心抓手

- 写写冲突才最敏感
- alignas(64) 只是常见策略，具体 cache line 以平台为准
- per-thread accumulation 是高效方案

工程 Gotcha

把 false sharing 与 data race 混为一谈。它可以完全没有竞态但仍严重掉性能。

高频追问树

- 原子变量为什么也可能 false share?
- 如何设计 job counter 避免热点 cache line?

面试官在听什么

是否能区分 correctness 与 performance concurrency bug。

第 3 章

操作系统、多线程与并发

本章目标

理解线程、同步、虚拟内存与任务系统，才能解释引擎为什么能稳定地把 CPU 吃满。

必须掌握：Job System；lock 选择；deadlock；acquire/release；虚拟内存；Game/Render/RHI 线程

Q21 • Process 与 Thread 的区别是什么？

难度： · 公开面经/官方资料可核验：[M2], [M3]

30 秒回答

进程拥有独立虚拟地址空间与资源命名空间；同一进程内线程共享大部分进程资源，但各自拥有栈、寄存器和执行上下文。

深入拆解

游戏引擎大多在单进程内用多线程共享大量状态，因为线程间传递指针和内存成本低；代价是同步复杂。跨进程工具（shader compiler worker、asset cooker、崩溃收集器）则通过隔离提高稳定性与安全性。面试中要能进一步讨论 IPC、调度单位和地址空间隔离，而不是只背“进程资源独立，线程共享资源”。

核心抓手

- 地址空间是核心区别
- 线程是调度执行流
- 进程隔离适合崩溃边界
- 共享内存带来同步责任

工程 Gotcha

把“线程一定比进程创建快”当跨平台绝对结论。

高频追问树

- 线程之间共享哪些资源？
- 进程间通信有哪些方式？
- 为什么 shader compiler 常做独立进程？

面试官在听什么

是否能把 OS 概念映射到真实引擎架构。

Q22 • Context Switch 为什么贵？

难度： · 公开面经/官方资料可核验：[M2]

30 秒回答

切换需要保存/恢复执行上下文，调度器参与，并可能破坏 cache/TLB/branch predictor 的局部性；真正损耗不只是几条寄存器指令。

深入拆解

大量细粒度线程会让系统花更多时间在调度和数据重新热身上。现代 Job System 通常使用固定 worker 数量，把工作拆成 task 而不是为每个任务新建线程。优化应从减少阻塞、降低 oversubscription、合理 task 粒度入手。若任务过细，scheduler overhead 也会超过实际工作。

核心抓手

- 区分 thread 与 task
- oversubscription 会增加调度
- cache/TLB 污染是隐藏成本
- 任务粒度需要 profiling

工程 Gotcha

回答“保存寄存器所以慢”，过于表面。

高频追问树

- 用户态协程能减少什么开销？
- 线程池 worker 数通常如何选？
- I/O worker 与 compute worker 是否应分开？

面试官在听什么

是否理解调度成本与任务系统设计。

Q23 · 什么是 Job System? 为什么比 “一个子系统一个线程” 更可扩展?

难度: · 公开面经/官方资料可核验: [01]

30 秒回答

Job System 把工作拆成大量任务, 交给固定 worker pool 动态调度; 不同系统不再永久绑定某个线程, 能通过 work stealing 改善负载不均。

深入拆解

Animation、visibility、physics island 等工作量每帧变化, 如果固定 “动画线程/物理线程”, 很容易一个线程满载而其他线程闲置。Task graph 通过依赖边表达先后关系, 例如 Skinning 依赖 Animation Pose, Render Extraction 依赖 Transform Update。高质量实现还会处理 task priority、fiber/continuation、等待时帮助执行其他 job, 避免 worker 阻塞。

核心抓手

- 固定 worker pool
- work stealing
- dependency DAG
- 等待策略比 “线程数量” 更关键

工程 Gotcha

把 task 拆得无限小。调度和队列开销会吞掉收益。

高频追问树

- Job 如何表示依赖?
- worker 等待一个 job 时为什么可以去执行别的 job?
- 如何避免主线程成为 submission bottleneck?

面试官在听什么

是否理解现代引擎 CPU 并行的核心抽象。

Q24 • Mutex、Spinlock、Semaphore 怎么选？

难度： • 高频能力重构

30 秒回答

Mutex 适合等待时间可能较长的互斥；Spinlock 适合极短临界区，因为等待线程会忙等；Semaphore 表示可用资源计数或并发配额，不等价于二元互斥。

深入拆解

正确选择取决于等待时间、竞争概率和调度成本。Spinlock 在单核或线程可能被抢占时尤其危险；持有 spinlock 做 I/O 更是灾难。Mutex 的实现也可能先短暂自旋再睡眠。Semaphore 常用于“最多 N 个资源/工作槽”。引擎中更重要的原则是减少共享可变数据，而非精细挑锁。

核心抓手

- 短临界区才适合 spin
- blocking primitive 会让出 CPU
- semaphore 表达数量
- 优先减少锁粒度和共享写

工程 Gotcha

认为 spinlock 一定比 mutex 快。高竞争下它可能浪费整核。

高频追问树

- 读写锁何时反而更慢？
- ticket lock 与普通 spinlock 区别？
- 优先级反转是什么？

面试官在听什么

是否基于 workload 做同步原语选择。

Q25 · 什么是 Deadlock? 四个必要条件是什么?

难度: · 高频能力重构

30 秒回答

经典必要条件是 mutual exclusion、hold-and-wait、no preemption、circular wait。工程上最常用的预防方式是固定锁顺序、缩短持锁时间和减少嵌套锁。

深入拆解

引擎复杂之处在于锁跨系统：Asset Manager、Render Thread、GC、Editor callbacks 可能形成很长依赖链。仅靠“发现死锁后加 timeout”不可靠。更好的策略是 ownership 分区、message passing、task dependency，或者调试版记录 lock-order graph。对 GPU fence 的等待也要警惕 CPU-GPU 循环依赖，虽然它不一定是传统 mutex deadlock。

核心抓手

- 四个必要条件
- lock ordering
- 避免持锁调用未知回调
- 异步系统也会形成等待环

工程 Gotcha

把 starvation 与 deadlock 混为一谈。

高频追问树

- 如何设计 lock hierarchy?
- 两个线程死锁时如何用 debugger 定位?
- Render Thread 等 Game Thread 是否可能形成死锁?

面试官在听什么

是否有系统级依赖图意识。

Q26 · 如何实现一个 SPSC 无锁环形队列?

难度: · 高频能力重构

30 秒回答

用固定 ring buffer 和两个单调递增/循环索引: producer 独占写 tail, consumer 独占写 head; 发布新 tail 用 release, 读取 tail 用 acquire, 从而保证 payload 可见。

深入拆解

SPSC 简单是因为每个索引只有一个写者, 不需要 CAS。producer 先写元素再 release-store tail; consumer acquire-load tail 后才能安全读元素。为了避免 head/tail false sharing, 可把它们放在不同 cache line。满/空判定可以保留一个槽位, 或使用额外 sequence。扩展到 MPMC 后会复杂很多, 需要 CAS、每槽 sequence 或其他算法。

核心抓手

- 单写者简化同步
- release/acquire 建立 happens-before
- ring buffer 避免 allocation
- 索引自身也要考虑 false sharing

工程 Gotcha

只写 `atomic<int>` 但仍用 `relaxed`, 导致另一线程看到 tail 更新却看不到 payload 写入。

高频追问树

- 为什么 SPSC 不需要 CAS?
- 如何区分 full 与 empty?
- MPMC 为什么更难?

面试官在听什么

是否真的理解 lock-free correctness 与 memory model。

Q27 • memory_order_relaxed / acquire / release 是什么？

难度： • 高频能力重构

30 秒回答

relaxed 只保证该原子操作本身原子，不建立跨变量顺序；release 发布此前写入，acquire 观察对应 release 后，建立 happens-before，使先前写对读取线程可见。

深入拆解

典型 producer-consumer：先写 payload，再 release-store ready；consumer acquire-load ready 后再读 payload。seq_cst 额外提供单一全局顺序，最易推理但可能限制优化。真正的面试重点是能否从一个小例子说明“为什么 relaxed 不够”，而不是背枚举。也要理解编译器重排与 CPU 重排都受 memory ordering 约束。

核心抓手

- 原子性与可见性/顺序性是不同概念
- release/acquire 常用于发布对象
- seq_cst 最强但不是默认答案
- 数据竞争仍是 UB

工程 Gotcha

认为 volatile 可以替代 atomic 或内存屏障。C++ volatile 不提供线程同步语义。

高频追问树

- acq_rel 用在哪些 RMW 操作？
- 什么是 release sequence？
- 为什么 x86 上有些 relaxed 看起来也“能跑”？

面试官在听什么

是否能进行跨线程 happens-before 推理。

Q28 • 虚拟地址如何转换成物理地址？Page Fault 发生了什么？

难度： · 公开面经/官方资料可核验：[M3]

30 秒回答

CPU 先用虚拟页号查询 TLB；miss 时走页表；若页表项表示页面不在内存或权限不符，会触发 page fault，由 OS 处理映射、加载或报错。

深入拆解

虚拟内存让每个进程看到独立地址空间，并支持 demand paging、memory mapping、copy-on-write。引擎的大型资产 streaming、mmap 文件、GPU upload staging 都会间接受 VM 行为影响。Major page fault 可能需要磁盘 I/O，延迟远高于普通 cache miss；因此实时程序会尽量预取和在加载阶段触页，避免 gameplay 突发 fault。

核心抓手

- TLB 是页表缓存
- page fault 不等于程序崩溃
- major/minor fault 成本不同
- 大页可降低 TLB 压力但有权衡

工程 Gotcha

把 page fault 只解释成“访问非法内存”。缺页加载也是正常 page fault。

高频追问树

- 多级页表为什么存在？
- Huge Page 有什么利弊？
- mmap 资产文件有什么优点？

面试官在听什么

是否理解操作系统内存机制对实时性能的影响。

Q29 • Game Thread 和 Render Thread 为什么要分开？

难度： · 公开面经/官方资料可核验：[O2], [O3]

30 秒回答

分线程可以让游戏逻辑与渲染命令构建并行，并让 Render Thread 维护自己的线程所有数据。关键是不要让 Render Thread 直接读 Game Thread 正在修改甚至可能释放的对象。

深入拆解

Unreal 官方明确描述 renderer 在独立线程运行，通常落后 Game Thread 一到两帧，并强调竞态难以复现。典型做法是 Game Thread 把可渲染状态复制成 Render Proxy/immutable snapshot，再异步 enqueue command；资源销毁用 fence 保证 Render Thread 已经停止使用。这样不仅是“更快”，更重要的是建立清晰 ownership。

核心抓手

- 线程所有权优先于到处加锁
- 通过 proxy/snapshot 跨线程传输状态
- 销毁需要 fence 或延迟回收
- 多帧延迟影响状态一致性

工程 Gotcha

让 Render Thread 保存指向 gameplay UObject 的裸指针并每帧读取。

高频追问树

- Game Thread 要销毁资源时怎么办？
- 为什么 Render Thread 可能落后两帧？
- RHI Thread 又解决什么？

面试官在听什么

是否能设计可证明安全的跨线程生命周期。

Q30 • Triple Buffering 与 Frames in Flight 有什么作用？

难度： · 公开面经/官方资料可核验：[O3]

30 秒回答

允许 CPU 构建后续帧时 GPU 仍处理前一帧，从而隐藏 CPU/GPU 延迟、提高吞吐；但排队太深会增加输入延迟和资源占用。

深入拆解

Frames in Flight 不等于交换链一定是三缓冲。通常每个 frame context 都有独立 command buffer、descriptor/constant allocation、fence。CPU 在重用某个 frame context 前等待其 fence，避免修改仍被 GPU 使用的数据。低延迟游戏可能限制 maximum pre-rendered frames，即使牺牲一点 GPU occupancy。

核心抓手

- 吞吐与 latency 是 trade-off
- 每帧资源必须独立或安全重用
- fence 定义重用安全点
- swapchain buffering 与 CPU frame context 是相关但不同概念

工程 Gotcha

认为“越多帧并行越快”。超过某点只会积累延迟。

高频追问树

- 双缓冲、三缓冲与 VSync 有什么关系？
- 如何测量 CPU queueing latency？
- 低延迟模式如何限制 frame queue？

面试官在听什么

是否理解异步流水线的吞吐/延迟权衡。

第 4 章

数据结构与算法

本章目标

算法题不仅考 LeetCode，也会被包装为空间查询、寻路、BVH 和实时系统场景题。

必须掌握：Top-K；A*；函数图；空间哈希；Octree/BVH；动态 BVH

Q31 · 10 万个数找最大的 1 万个，怎么做？

难度： · 高频能力重构

30 秒回答

流式场景维护大小 K 的小顶堆：前 K 个建堆，后续元素若大于堆顶则替换并下沉；复杂度 $O(N \log K)$ ，空间 $O(K)$ 。

深入拆解

如果数据全在内存且只需无序 Top- K ，可用 QuickSelect 平均 $O(N)$ 把第 K 大元素定位，再处理一侧；如果需要最终排序，则还需 $O(K \log K)$ 。面试应先问清楚 N 、 K 、是否流式、是否允许修改原数组、是否要有序输出。工程化时还要考虑 SIMD、并行 partition 或分块 local top- K 再 merge。

核心抓手

- 先澄清 streaming vs offline
- 小顶堆适合在线 Top- K
- QuickSelect 适合离线无序选择
- 最终排序是额外成本

工程 Gotcha

一上来就排序全部数据 $O(N \log N)$ ，没有利用 K 远小于 N 。

高频追问树

- 如果数据分布在多机怎么办？
- K 接近 N 时方案如何变化？
- 如何处理重复元素？

面试官在听什么

是否会从约束出发选算法，而不是背固定模板。

Q32 · 实现 A*，为什么它比 Dijkstra 更适合游戏寻路？

难度： · 高频能力重构

30 秒回答

A* 使用 $f(n)=g(n)+h(n)$ ，用启发函数把搜索集中到目标方向；当 $h=0$ 时退化为 Dijkstra。若 h 可采纳且满足相应条件，可保持最优性。

深入拆解

游戏地图中的核心是 heuristic。栅格上 Manhattan/Euclidean 距离要和允许移动方式匹配；过大的 h 会更贪心但可能失去最优性。open set 常用优先队列，closed set 记录已处理节点。工业实现还关心节点内存、重复 decrease-key、路径平滑以及 navmesh polygon graph 上的 funnel。

核心抓手

- $f=g+h$
- admissible heuristic 不高估真实剩余代价
- Dijkstra 是 $h=0$ 的特例
- 启发越强通常扩展节点越少

工程 Gotcha

只说“A* 更快”，不说明快来自搜索方向性，也不说明启发函数的正确性条件。

高频追问树

- consistent heuristic 是什么？
- open set 如何实现？
- NavMesh 上如何把 polygon path 转成平滑路径？

面试官在听什么

是否理解算法性质而非只会写模板。

Q33 • A* 在开放世界地图里为什么仍可能很慢？

难度： • 高频能力重构

30 秒回答

即使有启发函数，单层图规模太大时节点扩展、内存和动态障碍更新仍可能昂贵；需要层次化图、区域抽象、NavMesh hierarchy 或其他加速。

深入拆解

开放世界通常把路径问题拆成“宏观区域路径 + 局部精确路径”。Hierarchical Pathfinding 先在 sector/portal graph 上找远距离路线，再在局部 mesh 细化；多人单位同一目标可用 flow field；规则网格可考虑 Jump Point Search。实时游戏还常把搜索分帧执行，设置 budget，避免单帧 spike。

核心抓手

- 层次化减少搜索空间
- 批量单位可共享导航场
- 分帧 budget 避免 hitches
- 动态障碍常在局部重规划

工程 Gotcha

只换一个更快 priority queue，忽略问题规模。

高频追问树

- 如何做异步寻路？
- 导航数据如何流式加载？
- 动态 navmesh 更新如何避免全局重建？

面试官在听什么

是否能把算法放进实时帧预算与大世界架构。

Q34 • Bresenham 画线算法为什么能避免浮点数?

难度: • 高频能力重构

30 秒回答

它维护离散化误差项，在主轴每前进一步时判断是否需要在次轴也前进一步，把理想连续直线转换成整数增量更新。

深入拆解

以 $0 \leq \text{slope} \leq 1$ 为例，每次 $x++$ ，累计 dy ；误差超过阈值时 $y++$ 并减去 dx 。通过对八个象限做对称处理，可覆盖任意方向。该思想代表一种重要面试能力：把几何判定转成增量整数算法。虽然现代 GPU 不靠 Bresenham 实现所有细节，但它仍是经典 rasterization 入门题。

核心抓手

- 误差累积替代浮点斜率
- 每步只做整数加减与比较
- 通过象限归一化处理任意方向

工程 Gotcha

只背代码，不会解释 error term 的几何含义。

高频追问树

- 如何扩展到画圆?
- 为什么三角形 raster 更常用 edge function?

面试官在听什么

能否解释离散化算法背后的数学。

Q35 · 二叉树最近公共祖先怎么做？

难度： · 高频能力重构

30 秒回答

一般二叉树可递归：若当前节点是 p/q 就返回；分别搜索左右子树，若两边都非空则当前节点为 LCA，否则返回非空一侧。时间 $O(N)$ 。

深入拆解

如果是 BST，可利用有序性根据 p/q 值决定向左或右；如果有 parent pointer，可把问题变成两条祖先链求交；如果需要大量 LCA 查询，则可预处理 binary lifting 或 Euler Tour + RMQ。面试官常看你能否根据“单次/多次查询、树性质”选择不同方案。

核心抓手

- 一般树递归 $O(N)$
- BST 可利用排序性质
- 多查询考虑预处理
- 有 parent pointer 可转链表相交

工程 Gotcha

没确认 p 、 q 是否一定存在树中；某些题目需要额外存在性判断。

高频追问树

- 如果是 N 叉树呢？
- 一百万次 LCA 查询如何优化？

面试官在听什么

是否会根据问题模型变化推导方案。

Q36 · 最长合法括号如何做到 $O(N)$?

难度： · 高频能力重构

30 秒回答

栈方案以 -1 作为最近非法边界；遇左括号压索引，遇右括号弹栈，若栈空则当前索引成为新边界，否则长度为 `i-stack.top`。

深入拆解

这个技巧的本质是保存“当前合法段之前的最近无效位置”。也可用 DP: `dp[i]` 表示以 `i` 结尾的最长合法长度，并根据前一个匹配位置递推；还可左右两次扫描计数，用 $O(1)$ 空间。面试中若能给两种方案并比较，更容易体现算法理解。

核心抓手

- 栈保存索引而不是字符
- 哨兵 -1 统一边界
- DP/双向扫描是替代方案

工程 Gotcha

遇到栈空后忘记压入当前右括号索引，后续长度会跨过非法边界。

高频追问树

- 如何用 $O(1)$ 空间？
- 如果括号有三种类型怎么做？

面试官在听什么

是否理解边界状态设计。

Q37 · 所有节点出度 1，如何寻找最长环？

难度： · 公开面经/官方资料可核验：[M3]

30 秒回答

由于每个节点最多一个后继，图是若干“链进入环”的函数图。可用访问时间戳记录当前 DFS 路径，回到本轮路径节点时用时时间差得到环长；整体 $O(V)$ 。

深入拆解

另一种思路是 Kahn 式拓扑剥离：统计入度，不断删除入度为 0 的节点，剩余节点全部在环上，再遍历各环长度。时间同样 $O(V)$ ，实现简单且容易证明。该题很适合展示你能否利用“出度 1”这个特殊结构，而不是套一般图算法。

核心抓手

- 函数图结构
- 时间戳 DFS
- 入度剥离后剩余即环
- 整体 $O(V)$

工程 Gotcha

对每个起点重新 DFS，造成 $O(V^2)$ 。

高频追问树

- 如何同时返回环中的节点？
- 如果要求最小编号环怎么办？

面试官在听什么

能否抓住题目特殊约束降复杂度。

Q38 · 如何设计空间哈希解决大量对象邻域查询？

难度： · 高频能力重构

30 秒回答

把世界空间按固定 cell size 量化，位置映射到整数 cell 坐标，再哈希到 bucket；查询半径范围时只访问覆盖范围内的邻近 cell。

深入拆解

cell size 是关键超参数：太小会访问很多 cell，太大每个 bucket 对象过多。动态对象每帧只在跨 cell 时更新归属。对于 3D 稀疏世界，hash grid 比完整 3D array 节省空间；但极不均匀场景可能退化，需要层次结构或多级网格。Broad phase、粒子邻域、crowd simulation 都常用。

核心抓手

- 量化位置到整数 cell
- 查询只检查邻域 bucket
- cell size 决定候选数量
- 适合动态物体快速更新

工程 Gotcha

哈希只减少候选，不代表同 bucket 内对象一定相交，仍需 narrow phase。

高频追问树

- 如何避免负坐标取整错误？
- cell size 如何选？
- 与 BVH 比较动态更新成本。

面试官在听什么

是否能把数据分布与参数选择联系起来。

Q39 • Quadtree、Octree、BVH 有什么区别？

难度： · 公开面经/官方资料可核验：[M5]

30 秒回答

Quadtree/Octree 规则地划分空间；BVH 根据对象/几何构造包围体层级。前者适合空间查询与相对均匀分布，BVH 更贴合几何分布并广泛用于 ray tracing。

深入拆解

规则树的优势是位置天然决定节点，更新和邻域查询直观；缺点是空区域也可能产生层级且非均匀分布会造成深树。BVH 叶节点直接包围 primitive/object，节点边界可重叠，构建成本更高但 traversal 通常能快速排除大量几何。不能仅用“八叉树每层 8 个孩子所以一定更浅”判断性能。

核心抓手

- 空间划分 vs 对象层次
- BVH 节点可重叠
- 树深不是唯一性能指标
- 数据分布决定结构选择

工程 Gotcha

把 BVH 说成空间被严格切成不相交区域。

高频追问树

- KD-tree 与 BVH 呢？
- Nanite 为什么可能使用宽 BVH？
- 什么时候 loose octree 更适合动态物体？

面试官在听什么

是否理解空间数据结构的建模差异。

Q40 · 动态对象很多时 BVH 怎么更新？

难度： · 公开面经/官方资料可核验：[M2], [M5]

30 秒回答

常见策略是 refit 更新节点 bounding box, 成本低但树质量会逐渐恶化; 必要时局部/全局 rebuild。实践中常采用 refit + periodic rebuild 的混合策略。

深入拆解

如果拓扑不变、对象只轻微运动, bottom-up refit 很有效; 若对象跨越很大范围, 原本邻近的 leaf 已不再邻近, overlap 增加导致 traversal 变慢。可通过质量指标 (SAH cost、overlap ratio) 触发 rebuild, 或把动态/静态对象分成不同结构。硬件光追常见 TLAS 每帧更新、BLAS 对静态 mesh 复用。

核心抓手

- refit 便宜但会退化
- rebuild 恢复质量但昂贵
- 动静分离
- 用质量指标决定何时重建

工程 Gotcha

每帧无脑全重建, 或只 refit 永不监控退化。

高频追问树

- TLAS/BLAS 如何更新?
- 局部 rotation 优化 BVH 是什么?
- 如何并行 refit?

面试官在听什么

是否理解动态加速结构的长期性能。

第 5 章

3D 数学与几何

本章目标

坐标空间、四元数、法线变换、射线求交与透视插值，是所有渲染推导的数学地基。

必须掌握： MVP/clip/NDC；inverse-transpose normal；Quaternion/SLERP；frustum；ray-triangle；perspective-correct interpolation

Q41 · 游戏渲染中有哪些坐标空间？

难度： · 高频能力重构

30 秒回答

典型链路是 Local/Model $\rightarrow$ World $\rightarrow$ View $\rightarrow$ Clip $\rightarrow$ NDC $\rightarrow$ Screen。列向量约定下常写 $p_{clip} = P V M p_{local}$ ；具体乘法顺序必须与数学 convention 一致。

深入拆解

World 负责对象定位，View 把世界转换到相机坐标，Projection 把视锥映射到 clip space；之后做 clipping 与 perspective divide 得 NDC，再 viewport map 到屏幕。高级回答应明确 clip space 中 w 的作用、NDC z 范围在不同 API 的差异，以及 row-major/column-major memory layout 与数学上的 row/column vector 并非同一个概念。

核心抓手

- 明确各空间职责
- clip 发生在 perspective divide 之前
- API 的 NDC depth range 可能不同
- 矩阵存储布局与向量约定要分开

工程 Gotcha

背“模型-世界-观察-裁剪”但无法从一个点手推转换。

高频追问树

- 法线在哪个空间做光照最好？
- 为什么 clipping 在除 w 之前？
- Vulkan 与 OpenGL 的 NDC 有什么差异？

面试官在听什么

能否把矩阵、API convention 和实际 shader 串起来。

Q42 · 齐次坐标为什么需要第四维？

难度： · 高频能力重构

30 秒回答

把 3D 点扩展成 (x,y,z,w) 后，平移、投影等仿射/射影变换都能统一写成矩阵乘法；点通常 $w=1$ ，方向通常 $w=0$ 。

深入拆解

最重要的是 perspective projection：投影矩阵把与深度相关的量写入 w ，随后除以 w 产生远小近大的效果。齐次坐标还让无穷远方向、projective equivalence 和 clipping 有统一数学表达。面试回答不应停在“为了平移”，因为真正渲染价值在整个 projective pipeline。

核心抓手

- 平移可纳入矩阵
- 方向 $w=0$ 不受平移影响
- perspective divide 使用 w
- 同一欧氏点可由比例等价齐次坐标表示

工程 Gotcha

认为 w 永远等于 1。经过投影后 w 通常与 view-space depth 相关。

高频追问树

- 为什么方向向量 $w=0$ ？
- 投影矩阵中 w 是怎么来的？
- clip plane 如何用齐次坐标表示？

面试官在听什么

是否真正理解投影几何。

Q43 · 为什么 Normal 不能直接乘 Model Matrix?

难度： · 高频能力重构

30 秒回答

非均匀缩放会破坏“法线与切线垂直”关系，因此法线应乘线性变换的 inverse transpose: $n' = (M^{-1})^T n$ ，再归一化。

深入拆解

推导从约束 $n^T t = 0$ 出发。切线变换 $t' = M t$ ，希望 $n'^T t' = 0$ ，则 $n'^T M t = 0$ ，因此 $M^T n'$ 与 n 共线，得到 $n' = M^{-T} n$ 。若模型只有旋转与统一缩放，可简化；但工程上 shader 常直接准备 normal matrix。切记位置的 4x4 平移部分不应作用于方向。

核心抓手

- 从垂直约束推导 inverse transpose
- 非均匀缩放是关键反例
- 变换后需要 normalize
- 只取 3x3 线性部分

工程 Gotcha

死记公式但不知道何时可以直接用 model matrix。

高频追问树

- 切线 T 是否也要 inverse transpose?
- TBN 在非均匀缩放下如何保持正交?

面试官在听什么

是否会从几何不变量推导公式。

Q44 • Quaternion 为什么适合表示旋转？

难度： · 高频能力重构

30 秒回答

单位四元数用 4 个数表示 3D 旋转，组合是乘法，避免 Euler angle 的参数奇异；同时适合球面插值。

深入拆解

四元数并不是“完全没有奇异性”这类口号，它的核心优势是用单位 3-sphere 上的点表示 $SO(3)$ 的双覆盖，避免 Euler 三角参数化在某些姿态失去自由度。 q 与 $-q$ 表示同一旋转，因此插值前常需要检查 $\text{dot} < 0$ 时翻转一个 quaternion，选择最短弧。累计旋转后也要适当归一化抑制数值漂移。

核心抓手

- 单位四元数
- q 与 $-q$ 同一旋转
- 乘法表示旋转复合
- 插值前处理 shortest path

工程 Gotcha

说“四元数没有万向节锁”却无法解释 Euler 为什么会锁。

高频追问树

- 轴角如何转 quaternion？
- 为什么 quaternion 需要 normalize？
- 旋转顺序如何体现在乘法顺序？

面试官在听什么

是否理解旋转空间，而非只会调用库函数。

Q45 · SLERP 和 LERP 有什么区别?

难度: · 高频能力重构

30 秒回答

LERP 在四维坐标中直线插值, 归一化后可近似旋转插值; SLERP 沿单位四元数球面大圆插值, 保持恒定角速度。

深入拆解

SLERP 公式包含 $\sin((1-t)/\sin)$ 与 $\sin(t)/\sin$ 。很小时直接公式数值不稳定, 工程中通常退化为 NLERP。游戏动画大量使用 NLERP, 因为更快且视觉上足够; 只有需要严格恒定角速度或较大角度时才必须 SLERP。若 $\text{dot}(q_0, q_1) < 0$, 应翻转其中一个四元数避免走长弧。

核心抓手

- SLERP 恒定角速度
- NLERP 快且常够用
- 小角度做线性近似
- 处理 q 与 $-q$ 双覆盖

工程 Gotcha

所有旋转插值都无脑 SLERP, 忽略性能与近似。

高频追问树

- NLERP 有什么误差?
- 动画 blend 多个 quaternion 怎么办?
- dual quaternion 又解决什么?

面试官在听什么

是否会从视觉需求和成本选择插值方法。

Q46 · 如何判断一个点/包围体是否在视锥体中？

难度： · 高频能力重构

30 秒回答

把视锥表示成六个平面。点对每个平面计算有符号距离；若在任意一个外侧则可剔除。Sphere/AABB 则用半径或投影半径做保守平面测试。

深入拆解

AABB 测平面可选最远/最近顶点，或计算 `center-extents` 在平面法线上的投影半径 $r = \text{dot}(\text{abs}(n), \text{extents})$ 。如果 $\text{center distance} + r < 0$ ，盒子完全在外。引擎会先用层次结构把成千上万对象聚合剔除，而不是逐对象逐平面测试到底。GPU-driven 场景还可能把 frustum test 放 compute shader。

核心抓手

- 六平面测试
- AABB 用 `center-extents` 高效测试
- 保守剔除不能产生 false negative
- 层次剔除减少测试数量

工程 Gotcha

对 AABB 只测试 8 个角点，虽然正确但没有利用结构。

高频追问树

- 如何从 VP 矩阵提取 frustum planes?
- OBB 怎么测?
- 为什么 near plane 对精度很重要?

面试官在听什么

是否能把数学测试优化成工程可用版本。

Q47 · Ray 与 Triangle 如何求交?

难度: · 高频能力重构

30 秒回答

Möller-Trumbore 把 Ray 与三角形的重心参数联立, 直接求 t, u, v ; 检查 $t \geq 0$ 、 $u \geq 0$ 、 $v \geq 0$ 、 $u+v \leq 1$ 即可判断命中。

深入拆解

算法避免显式计算三角形平面方程, 核心用叉乘/点乘解一个 3×3 线性系统。实现需处理 determinant 接近 0 的平行情况、back-face culling 选项和 epsilon。批量 ray tracing 中, ray-triangle 本身往往不是最大成本, 真正关键是 BVH traversal 能否减少绝大多数 triangle tests。

核心抓手

- Ray: $O+tD$
- 重心坐标 u/v
- determinant 判断平行
- epsilon 与浮点鲁棒性

工程 Gotcha

用 $==0$ 判断 determinant, 容易受浮点误差影响。

高频追问树

- 如何返回 barycentric attribute?
- 是否需要 normalize ray direction?
- watertight ray-triangle 算法解决什么?

面试官在听什么

是否理解几何求交的数值问题。

Q48 • AABB 和 OBB 的区别?

难度: • 高频能力重构

30 秒回答

AABB 与世界轴对齐, 存储/更新/测试快但旋转后包围会变松; OBB 随对象方向, 通常更紧但相交测试昂贵。

深入拆解

AABB-AABB 只需比较每个轴的区间; OBB-OBB 常用 Separating Axis Theorem, 在 3D 中需要检查两盒自身轴以及轴叉乘产生的候选分离轴。碰撞系统通常 broad phase 用 AABB, narrow phase 再用精确形状/OBB, 因为 broad phase 更重视快速拒绝而非最紧包围。

核心抓手

- AABB 便宜且适合 broad phase
- OBB 更紧但更新/测试贵
- SAT 用分离轴判断凸体不相交

工程 Gotcha

认为 OBB “一定更好”, 忽略它的 CPU 成本。

高频追问树

- 旋转 AABB 怎么快速更新?
- Capsule 为什么适合角色?
- swept AABB 用于什么?

面试官在听什么

是否会根据碰撞阶段选择包围体。

Q49 · 什么是重心坐标？

难度： · 高频能力重构

30 秒回答

三角形内任意点可写成 $P = A + B + C$ ，且 $+ + = 1$ ；三者可用于点内判定和顶点属性插值。

深入拆解

Rasterization 中屏幕位置可由 edge function 得到 barycentric weights，再插值颜色、法线、UV 等。对纹理坐标等来自 3D 的属性，屏幕空间权重不能直接线性插，需要透视矫正。重心坐标也可用于 ray-triangle 命中后插值 vertex attributes。

核心抓手

- $+ + = 1$
- 三者非负表示点在三角形内
- 可插值任意顶点属性
- 屏幕空间属性需区分是否 perspective-correct

工程 Gotcha

认为所有 attribute 都能直接用屏幕重心坐标线性插。

高频追问树

- edge function 如何得到 barycentric?
- 三角形外部时重心坐标有什么特征?

面试官在听什么

是否能把几何坐标与 raster attribute interpolation 对上。

Q50 · 为什么纹理属性必须做透视矫正插值？

难度： · 公开面经/官方资料可核验：[M3]

30 秒回答

因为 perspective divide 是非线性的，屏幕空间线性插值不对应 3D 空间中的线性变化。正确方法是插值 $\text{attribute}/w$ 和 $1/w$ ，再做除法恢复。

深入拆解

设顶点属性 a_i 与 clip w_i ，屏幕重心权重 $_i$ 下，正确 $a = (\sum _i a_i/w_i)/(\sum _i 1/w_i)$ 。直观上，远处顶点在屏幕上被压缩，如果直接线性插 UV，会出现纹理“游动/扭曲”。硬件 rasterizer 会自动完成 perspective-correct interpolation；shader 可用 noperspective 等修饰符选择不同规则。

核心抓手

- 透视投影破坏屏幕空间线性关系
- 插值 a/w 与 $1/w$
- 硬件默认通常是 perspective-correct
- 某些屏幕空间量反而需要 noperspective

工程 Gotcha

会背公式但不知道 w 来自 clip-space projection。

高频追问树

- 哪些 attribute 适合 noperspective？
- 中心采样、centroid interpolation 有什么区别？

面试官在听什么

能否从投影变换推导 raster interpolation。

第 6 章

实时渲染管线

本章目标

从 draw submission 到 raster、depth、lighting，要求能把 GPU 流水线与性能瓶颈对应起来。

必须掌握：完整 raster pipeline；Early-Z/Hi-Z；Forward/Deferred/Clustered；GBuffer 重建；instancing

Q51 • 请完整讲一遍现代 GPU Rasterization Pipeline。

难度： • 公开面经/官方资料可核验：[M1], [M2], [M3], [M5]

30 秒回答

从 CPU command submission 开始，经 vertex/mesh processing、clipping、perspective divide、viewport、rasterization、fragment shading、depth/stencil、blend，最终写 framebuffer；现代 API 还要考虑 indirect draw、async compute 和 resource barriers。

深入拆解

优秀回答应区分“固定功能阶段”和“可编程阶段”，并能指出数据在每步发生什么：vertex shader 产出 clip-space position；clipping 在齐次空间处理；rasterizer 生成 sample coverage 与插值；fragment shader 计算颜色/深度；ROP 阶段执行 depth/stencil/blend。更高阶还要谈 mesh shader、tile-based GPU 和 CPU submission overhead。

核心抓手

- 能按数据流而非背名称解释
- 明确 clip-space 与 raster 的边界
- 知道 Early-Z 可能提前 depth test
- 把 CPU/RHI submission 纳入完整 frame pipeline

工程 Gotcha

漏掉 clipping/perspective divide，或者把 rasterization 等同于 fragment shader。

高频追问树

- Tessellation 在哪里？
- Geometry Shader 为什么现在较少使用？
- Mesh Shader 改变了哪部分管线？
- draw call 为什么是 CPU 问题？

面试官在听什么

这是“能否从整体解释 renderer”的第一核心题。

Q52 • Rasterization 本质上在做什么？

难度： · 高频能力重构

30 秒回答

把连续几何 primitive 转换为离散 framebuffer sample 的 coverage，并为覆盖 sample 生成插值属性。

深入拆解

三角形 raster 常用 edge function 判断 sample 是否在三条边内，同时根据 signed area 得到 barycentric weights。硬件通常以 2x2 quad 组织 fragment 执行，以支持 derivative/ddx/ddy 和纹理 LOD，这会产生 helper lanes。MSAA 进一步把 coverage sample 与 shading sample 分开，因此“一个 pixel 就执行一次 shader”并非严格成立。

核心抓手

- coverage 与 shading 是两个概念
- edge function/重心坐标
- fragment 常以 quad 执行
- MSAA 对 coverage 与 shading 分离

工程 Gotcha

把 rasterization 简化成“把三角形变成像素”，没有讨论 sample/coverage。

高频追问树

- Top-left rule 是什么？
- 为什么 GPU 需要 2x2 quad？
- derivative 如何影响 texture LOD？

面试官在听什么

是否理解 rasterizer 的硬件执行模型。

Q53 • Vertex Shader 与 Fragment Shader 谁调用次数更多?

难度: • 高频能力重构

30 秒回答

没有绝对答案。典型高分辨率/高 overdraw 场景 fragment workload 更大; 高几何密度、复杂 vertex deformation 或 Nanite 类场景也可能 geometry/vertex 侧很重。

深入拆解

判断必须看 profiler: vertex count、post-transform cache 命中、screen coverage、overdraw、pixel shader 指令/纹理访存。一个 1 像素三角形可能让 vertex 开销远高于有效 pixel; 反过来 fullscreen post pass 几乎没有 vertex 成本却有数百万 fragment。优化前先确定 geometry-bound、pixel-bound、bandwidth-bound 或 submission-bound。

核心抓手

- 不要用调用次数单独判断成本
- 关注每次 shader cost
- 小三角形会浪费 raster efficiency
- overdraw 放大 pixel workload

工程 Gotcha

回答 “Fragment Shader 一定更多”。

高频追问树

- 如何证明当前是 pixel-bound?
- 小三角形为什么伤性能?
- post-transform vertex cache 有什么作用?

面试官在听什么

是否会用 workload 数据而非经验口号。

Q54 · 什么是 Early-Z? 它为什么能加速?

难度: · 公开面经/官方资料可核验: [M2], [M4], [M6]

30 秒回答

在昂贵 fragment shading 之前先做深度测试, 若 fragment 必然被更近几何遮挡, 就直接拒绝, 从而避免无效 pixel shader 工作。

深入拆解

Early-Z 是概念, 不应假设硬件严格按一个固定阶段实现。GPU 可能维护 hierarchical depth, 在 primitive 或 tile 级提前 reject。收益取决于 overdraw 和 fragment cost; 前向渲染常通过 front-to-back 排序提高已有深度的有效性。若 shader 有影响深度/可见性的副作用, 硬件必须保守处理。

核心抓手

- 目标是避免无效 fragment work
- Hi-Z 可在更粗粒度拒绝
- front-to-back 增强 early reject
- 实际实现由 GPU 决定

工程 Gotcha

说“Early-Z 在 fragment shader 前固定有一个阶段”, 过度简化硬件。

高频追问树

- Depth Prepass 与 Early-Z 是一回事吗?
- 透明物体为什么不同?
- 移动 TBDR 下 Early-Z 有什么特殊性?

面试官在听什么

是否从可见性与成本理解深度优化。

Q55 • Early-Z 什么时候可能失效或受限？

难度： · 公开面经/官方资料可核验：[M4], [M6]

30 秒回答

当 fragment shader 可能改变最终深度或可见性，或存在需要保留副作用的写操作时，GPU 不能随意在 shader 前丢弃该 fragment。

深入拆解

典型因素包括写 `gl_FragDepth/SV_Depth`、某些 `discard/alpha test` 路径以及 UAV/storage side effects。不同 GPU/API 会采用 early/late test 的保守规则，也可能启用 `early_fragment_tests` 等显式语义。重点不是背一份“失效清单”，而是推理：如果提前 kill 会改变程序可观察结果，就不能安全提前。

核心抓手

- 从程序副作用推理是否允许 early reject
- 写深度是最直接限制
- `discard` 的实际影响依架构/编译器而异
- 显式 early tests 也要理解语义

工程 Gotcha

把 alpha test 简单说成“必然关闭 Early-Z”。现实更细，需要看硬件与 shader 行为。

高频追问树

- 为什么写 UAV 会影响 early test?
- conservative depth 有什么用?
- `early_fragment_tests` 改变哪些语义?

面试官在听什么

是否能用 correctness 推导 GPU 优化条件。

Q56 • Hi-Z / Hierarchical Z 是什么？

难度： · 公开面经/官方资料可核验：[M2]

30 秒回答

把深度缓冲构成层次化的 min/max 表示，让 GPU 或 compute culling 在粗粒度先判断一块区域是否完全被遮挡，避免逐像素/逐三角形测试。

深入拆解

用于 occlusion culling 时，先把对象 AABB 投影到屏幕，选合适 mip 的 Hi-Z 深度做保守比较；如果整个对象的最近深度仍在已有遮挡之后，就可以剔除。Reverse-Z 时层次聚合规则要相应变化。Hi-Z 也用于 SSR、ray marching 和 software occlusion。关键是保守：宁可多画，不能错误剔除可见对象。

核心抓手

- 深度金字塔
- 粗粒度保守拒绝
- 选择 mip 与屏幕包围范围相关
- Reverse-Z 改变 min/max 语义

工程 Gotcha

用单个中心采样判定整个对象遮挡，可能产生错误剔除。

高频追问树

- 如何处理上一帧 Hi-Z 的 temporal occlusion?
- occlusion culling 的 false negative 为什么危险?

面试官在听什么

是否理解“层次保守测试”的工程原则。

Q57 • Forward 和 Deferred Rendering 如何比较?

难度: • 公开面经/官方资料可核验: [M1], [M6]

30 秒回答

Forward 在几何 pass 直接完成 lighting, 透明/MSAA 友好; Deferred 先写 GBuffer 再按屏幕像素做 lighting, 适合大量动态光, 但带来显存/带宽和透明处理成本。

深入拆解

复杂度不应粗暴写成“Forward= 物体 $\times$ 灯, Deferred= 像素 $\times$ 灯”。现代 Forward+ 也会先做 light culling; Deferred 的 GBuffer 格式、材质模型数量和带宽往往才是核心成本。Deferred 对 alpha blending 透明通常另走 forward pass; MSAA 则可能让每 sample 的 GBuffer/lighting 昂贵。移动 TBDR 上 bandwidth 结构又不同。

核心抓手

- Forward 的材质自由度与透明性好
- Deferred 解耦几何与多灯 lighting
- GBuffer bandwidth 是核心成本
- 现代方案有 Forward+/Clustered

工程 Gotcha

把 Deferred 说成“灯光数量完全不影响性能”。

高频追问树

- 半透明怎么做?
- Deferred 多 shading model 如何编码?
- 移动端为什么常偏 Forward/Deferred hybrid?

面试官在听什么

是否能基于 workload 做渲染路径选择。

Q58 • Forward+ / Clustered Rendering 为什么出现?

难度: • 高频能力重构

30 秒回答

先把屏幕/视锥切成 tile 或 3D cluster, 为每个区域建立影响它的 light list, shading 时只遍历局部灯光, 从而让 Forward 也能支持大量动态光。

深入拆解

Tiled Forward 主要按 2D screen tile 划分, Clustered 再加入深度维度, 更好处理光在深度上的局部性。light culling 常用 compute shader, 构建 compact index list。工程难点包括 cluster 尺寸、最坏 light count、内存分配和透明物体复用。它代表一种普遍思想: 先做粗粒度空间分类, 再减少每像素循环。

核心抓手

- 空间分桶 light list
- Clustered 引入 depth slice
- compute prepass + forward shading
- 适合透明与 MSAA

工程 Gotcha

认为它只是 “Deferred 的另一种名字”。

高频追问树

- cluster z slice 如何划分?
- light list overflow 怎么处理?
- 为什么 logarithmic depth slice 常见?

面试官在听什么

是否理解现代 forward renderer 的数据结构。

Q59 · 为什么 Deferred GBuffer 可以不保存 World Position?

难度： · 公开面经/官方资料可核验：[M6]

30 秒回答

屏幕 UV + depth + inverse projection/view 足以重建 view/world position, 因此可以省掉高带宽 position render target。

深入拆解

从 NDC 构造 clip 坐标, 把 depth 放到正确 API 范围, 再乘 inverse projection 得 view position, 做齐次除法; 需要 world position 时再乘 inverse view。更高效做法可以用 view ray 与线性深度重建, 避免完整矩阵。注意硬件 depth 常是非线性且可能使用 Reverse-Z, 必须按实际 projection 反推。

核心抓手

- Depth 保存了沿 view ray 的位置信息
- 重建需尊重 API depth convention
- Reverse-Z/无限远投影会改变公式
- 省 RT 意味着省带宽

工程 Gotcha

直接把 depth 当线性 view-space z。

高频追问树

- 如何线性化 depth?
- 为什么 position RT 很贵?
- 法线能不能也从 depth 重建? 有什么质量问题?

面试官在听什么

是否能从数据冗余和带宽角度优化 GBuffer。

Q60 • Instancing 为什么能减少 Draw Call?

难度： • 高频能力重构

30 秒回答

把共享 mesh/material 状态的一批实例用一次 draw 提交，并用 instance buffer/ID 提供每实例 transform、颜色或 material index，减少 CPU API submission 与 state validation。

深入拆解

Instancing 的收益取决于 CPU 是否 submission-bound。若实例只几个或 shader 极重，draw call 减少未必改善帧时。过度合批也可能降低 culling 粒度：一个超大 batch 只要部分可见就全部提交。现代 GPU-driven 进一步把可见实例 compact 后用 indirect draw，解决 CPU 合批与可见性之间的矛盾。

核心抓手

- 共享 geometry/state
- per-instance data
- 减少 CPU submission
- 合批与 culling 粒度存在 trade-off

工程 Gotcha

把 Instancing 当作“GPU 顶点只处理一次”。每个实例仍需要独立 transform 后的顶点处理。

高频追问树

- Instancing 与 static batching 区别?
- 如何做 indirect instancing?
- 大量不同材质怎么合批?

面试官在听什么

是否理解 draw call 成本到底来自哪里。

第 7 章

PBR、阴影、GI 与 Ray Tracing

本章目标

从 Rendering Equation 出发，把 BRDF、软阴影、Monte Carlo 与 BVH 串成统一物理图景。

必须掌握：Rendering Equation；Cook-Torrance；normal map/TBN；shadow；PCSS；Monte Carlo/MIS；BVH

Q61 · 写出 Rendering Equation，并解释每一项。

难度： · 公开面经/官方资料可核验：[M4]

30 秒回答

$L_o(x, w_o) = L_e(x, w_o) + \int_{\Omega} f_r(x, w_i, w_o) L_i(x, w_i) (n \cdot w_i) d\omega_i$ 。它把“自发光 + 所有入射方向经材质散射后的贡献”统一起来。

深入拆解

L_o/L_i 是 radiance； f_r 是 BRDF，描述单位入射 irradiance 如何变成出射 radiance；cosine 项来自投影面积。递归关系在于 L_i 往往来自其他表面的 L_o ，这就是全局光照。实时渲染的各种技巧，本质上都在近似这个积分：direct lighting 只取显式光源，IBL 把环境积分预计算，path tracing 用 Monte Carlo 随机估计。

核心抓手

- radiance 是核心传输量
- BRDF + cosine
- L_i 递归来自场景
- 各种 GI/IBL/PT 都是方程的不同近似

工程 Gotcha

只会写公式，不会解释每一项单位与物理意义。

高频追问树

- 为什么积分只在半球？
- BRDF 需要满足哪些物理性质？
- irradiance 与 radiance 区别？

面试官在听什么

这是判断图形学是否形成统一理论框架的核心题。

Q62 • Cook-Torrance BRDF 怎么理解?

难度: • 公开面经/官方资料可核验: [M1], [M2], [M3]

30 秒回答

微表面镜面项常写 $f_s = D(h)F(v,h)G(l,v,h) / [4(n \cdot l)(n \cdot v)]$; D 描述微表面法线分布, F 是 Fresnel, G 是遮蔽/阴影。

深入拆解

D 决定高光形状, 实时 PBR 常用 GGX; F 表示入射越掠射反射越强, 常用 Schlick 近似; G 修正微表面之间的 masking-shadowing。分母来自微表面几何与变量变换。完整材质通常还需要 diffuse term, 并满足能量守恒。面试应能解释 roughness 改变的是微表面分布, 而不是简单“降低高光亮度”。

核心抓手

- D/F/G 各有明确物理角色
- GGX 常见于实时渲染
- Schlick Fresnel 是高效近似
- roughness 会让高光更宽更低

工程 Gotcha

把 metallic 当“高光强度滑杆”或把 roughness 当“模糊贴图”。

高频追问树

- GGX 为什么比 Blinn-Phong 更符合长尾高光?
- Smith G 如何工作?
- 能量守恒如何处理 diffuse/specular?

面试官在听什么

是否理解现代 PBR 的微表面模型。

Q63 • Metallic/Roughness PBR Workflow 是什么意思？

难度： · 公开面经/官方资料可核验：[M2]

30 秒回答

它用 BaseColor、Metallic、Roughness 等参数约束材质到可解释物理范围：dielectric F0 常约 0.04，metal 的 base color 主要进入 specular，diffuse 基本消失。

深入拆解

这种 workflow 的价值是“标准化材质空间”，让艺术资产在不同灯光环境下保持一致。Metallic 通常应接近 0 或 1，灰区主要用于混合边界；Roughness 改变 NDF，控制微表面方向分散度。AO 常只影响间接环境项，不能随意乘到所有 direct lighting。

核心抓手

- dielectric/metal 的光学差别
- roughness 控制微表面统计
- metal 无常规 diffuse
- workflow 让材质跨场景一致

工程 Gotcha

把 AO 直接乘最终颜色、把 metalness 当任意连续“金属感”。

高频追问树

- Specular/Glossiness workflow 与它如何转换？
- F0 为什么不是所有绝缘体都严格 0.04？

面试官在听什么

是否理解 PBR 参数背后的物理约束。

Q64 • Normal Map 为什么通常存在 Tangent Space?

难度： · 公开面经/官方资料可核验：[M2], [M3]

30 秒回答

Tangent-space normal 与模型表面局部 UV 基一起定义，因此同一法线纹理可以复用到不同位置/朝向和动画后的表面；shader 用 TBN 把贴图法线转到目标空间。

深入拆解

T 通常来自 position/UV 导数，B 可通过 handedness 与 $\text{cross}(N, T)$ 重建；必须注意模型导入工具与 shader 的 tangent basis 约定一致，否则会出现接缝或高光翻转。采样 normal map 后还要把 $[0,1]$ 解码到 $[-1,1]$ 并 normalize。非均匀缩放下 TBN 的正交性也需要谨慎维护。

核心抓手

- TBN 是局部坐标基
- bitangent 常由 sign 重建
- tangent basis 必须与烘焙工具一致
- 法线纹理解码后归一化

工程 Gotcha

只会写 $TBN * n$ ，却不知道 tangent 如何从 UV 求。

高频追问树

- MikkTSpace 为什么重要？
- 镜像 UV 如何处理 handedness？
- object-space normal map 有什么优缺点？

面试官在听什么

是否能连接 asset pipeline 与 shader 数学。

Q65 • Shadow Mapping 原理是什么?

难度: • 公开面经/官方资料可核验: [M1], [M2], [M6]

30 秒回答

先从光源视角渲染深度图;主视角 shading 时把世界点变换到 light clip/texture space, 与 shadow map 中最近深度比较, 判断是否被遮挡。

深入拆解

它是“可见性查询”的离散近似。关键问题包括 light projection、depth precision、bias、filtering、级联和 atlas。Directional light 的大场景常用 CSM 把相机 frustum 分段, 每段使用不同 shadow map; Point light 可能用 cubemap。Shadow map 的纹理分辨率和投影范围共同决定 texel footprint。

核心抓手

- 两 pass: light depth + camera compare
- shadow 是可见性而非“画黑色”
- 精度和投影范围决定质量
- 大范围需 CSM/VSM 等扩展

工程 Gotcha

忘记做 perspective divide/texture-space remap, 或认为 shadow map 保存世界距离而非投影深度。

高频追问树

- CSM split 怎么选?
- 点光源 shadow 怎么做?
- VSM/EVSM 与普通 depth compare 有何区别?

面试官在听什么

是否理解 shadow 系统是空间采样问题。

Q66 • Shadow Acne 和 Peter Panning 为什么出现?

难度: • 公开面经/官方资料可核验: [M1]

30 秒回答

离散 shadow depth 与浮点/采样误差使表面自比较时产生假阴影; 加 bias 可缓解, 但 bias 过大让阴影与物体分离, 形成 Peter Panning。

深入拆解

常见 bias 包括 constant depth bias、slope-scaled bias 和 normal offset。斜表面尤其容易出现 depth gradient 误差, 因此 slope bias 根据表面倾斜程度增加偏移。PCF/软阴影扩大采样范围后 bias 需求也可能变化。没有一个固定 bias 对所有 cascades/距离都最佳, 常需按 texel world size 缩放。

核心抓手

- acne 是自阴影误判
- bias 是精度补偿
- slope/normal bias 各有用途
- bias 与 shadow texel size 应关联

工程 Gotcha

只把 bias 数值调大直到 acne 消失, 导致远距离阴影漂浮。

高频追问树

- receiver-plane depth bias 是什么?
- Reverse-Z shadow map 会改变 bias 方向吗?

面试官在听什么

是否能把视觉伪影追溯到采样/精度。

Q67 • PCF 和 PCSS 有什么区别？

难度： · 公开面经/官方资料可核验：[M6]

30 秒回答

PCF 对邻域多个深度比较结果做平均，模糊硬 shadow edge；PCSS 先搜索 blocker，估计 penumbra 大小，再用与软影宽度相关的可变半径 PCF，近似 area light “近硬远软”。

深入拆解

PCF 不是对深度本身简单 blur，而是对 comparison result/filterable depth representation 做过滤。PCSS 的 blocker search 与 filter 都有多次采样，成本高且容易噪声/抖动，实际会用 Poisson disk、rotated pattern、temporal filtering。对于大面积软影，现代引擎也可能使用 VSM、ray traced shadow 等方案。

核心抓手

- PCF = 固定/有限 filter kernel
- PCSS = blocker search + penumbra estimate + variable PCF
- 采样模式影响噪声与条纹

工程 Gotcha

说 PCSS 只是“更大的 PCF”。它的核心是根据 blocker distance 改变半影宽度。

高频追问树

- 为什么 PCF 采样太规则会出现 banding？
- Poisson disk 解决什么？
- VSM 为什么可以线性过滤？

面试官在听什么

是否理解软阴影的几何来源。

Q68 • Path Tracing 为什么会有噪声？收敛速度如何理解？

难度： • 公开面经/官方资料可核验：[M4]

30 秒回答

Path Tracing 用 Monte Carlo 随机估计高维光传输积分。估计量方差导致图像噪声；独立样本标准误差通常按 $1/\sqrt{N}$ 下降，所以想把噪声标准差减半约需 4 倍样本。

深入拆解

路径追踪每个像素随机采样方向、光源和路径长度。无偏意味着期望值正确，不等于有限样本没有误差；很多工业技巧会接受轻微偏差换更低方差。Russian Roulette 用概率终止长路径同时保持期望；denoising 则利用空间/时间先验在少样本下重建。核心是“方差管理”，不是单纯增加 spp。

核心抓手

- Monte Carlo 方差来源
- 标准误差 $\sim 1/\sqrt{N}$
- 无偏与低方差是不同目标
- Russian Roulette 保持期望同时控制路径长度

工程 Gotcha

认为“无偏 = 每一帧都是正确答案”。

高频追问树

- 为什么 caustics 难收敛？
- Russian Roulette 权重如何补偿？
- temporal accumulation 为什么会 ghosting？

面试官在听什么

是否理解随机积分而非把 Path Tracing 当黑盒。

Q69 · 什么是 Importance Sampling?

难度： · 公开面经/官方资料可核验：[M4]

30 秒回答

在积分贡献大的区域更高概率采样，并用 $1/p(x)$ 校正权重；理想 proposal 与 $|f(x)|$ 成比例，可显著降低 Monte Carlo 方差。

深入拆解

渲染里常见 cosine-weighted hemisphere、BRDF sampling、light sampling。单一分布无法同时擅长所有情况：小而亮的光源适合 light sampling，强镜面材质适合 BRDF sampling，因此 Multiple Importance Sampling 用权重组合多个 proposal，降低“某一路径策略几乎采不到关键贡献”的风险。

核心抓手

- 改变采样分布必须除以 PDF
- 重要性采样目标是降方差
- 不同 proposal 擅长不同光路
- MIS 是现代 path tracing 核心

工程 Gotcha

只说“亮的地方多采样”，但没说明 PDF 补偿，否则估计会产生偏差。

高频追问树

- 如何从 CDF 采样?
- Balance/Power heuristic 是什么?
- 环境贴图如何做 importance sampling?

面试官在听什么

是否具备概率与渲染结合的能力。

Q70 • BVH 在 Ray Tracing 中解决什么问题？

难度： · 公开面经/官方资料可核验：[M2], [M5]

30 秒回答

把大量 primitive 组织成层次包围体，ray 先与少量 AABB 相交测试，只进入可能命中的子树，把暴力 $O(N)$ triangle test 降成远少的候选。

深入拆解

BVH 质量受构建策略影响。SAH 用表面积近似 ray 命中概率，平衡 traversal cost 与 leaf primitive cost；实时构建可用 LBBVH/HLBBVH 等更快方法。硬件 RT 常把 mesh 的几何结构作为 BLAS，再用 TLAS 组织实例，这样实例 transform 改变时可主要更新 TLAS。Traversal 性能还受 node width、memory layout 和 ray coherence 影响。

核心抓手

- AABB reject 减少 triangle tests
- SAH 是质量目标
- BLAS/TLAS 分离 mesh 与 instance
- 内存布局和 ray coherence 影响 traversal

工程 Gotcha

只说“BVH 查询 $O(\log N)$ ”。BVH traversal 并没有严格平衡树的最坏复杂度保证。

高频追问树

- SAH 公式的直觉是什么？
- BVH4/BVH8 为什么可能更适合 SIMD/GPU？
- 动态场景怎么 refit？

面试官在听什么

是否理解加速结构的构建-遍历 trade-off。

第 8 章

引擎架构与资源系统

本章目标

引擎不是模块列表，而是生命周期、线程所有权、数据血缘、资源依赖和跨平台抽象的集合。

必须掌握：模块依赖；serialization schema；asset cook；GUID；streaming；Render Thread/RHI；Render Graph；RHI

Q71 · 如果从零设计游戏引擎，最核心模块有哪些？

难度： · 公开面经/官方资料可核验：[M1], [O1]

30 秒回答

至少需要 Platform/Core、Memory/Jobs、Filesystem/Asset、Reflection/Serialization、World/Scene、Rendering/RHI、Physics、Animation、Audio/Input、Scripting、Editor/Profiler 等，但真正关键是依赖方向和生命周期。

深入拆解

“列模块”只是入门。系统设计要回答：谁依赖谁？哪些模块可离线？runtime 是否必须知道 editor？跨平台层在哪里？资源 ID 与文件路径如何解耦？线程所有权如何划分？常见原则是底层 core 不依赖高层 gameplay，RHI 不感知具体游戏对象，Editor 通过反射/命令接口操作 runtime 数据。模块边界决定后续能否热重载、测试和跨平台。

核心抓手

- 先定义 dependency direction
- offline pipeline 与 runtime 解耦
- 平台抽象在底层
- Profiler 是核心基础设施而非附加功能

工程 Gotcha

画一个“所有模块互相调用”的大图，没有依赖规则。

高频追问树

- 谁拥有 World?
- Renderer 是否能直接访问 Entity?
- 如何做 plugin/module hot reload?

面试官在听什么

是否具备架构边界意识，而不是功能清单思维。

Q72 • Scene Graph 和 ECS 有什么关系？

难度： • 高频能力重构

30 秒回答

Scene Graph 主要表达层次 transform/可见性关系，ECS 主要表达数据组合与批处理；它们解决不同问题，可以同时存在。

深入拆解

Transform hierarchy 需要 parent-child 更新顺序和局部/世界矩阵传播；ECS archetype 则希望相同组件连续存储。强行把所有 parent-child 都映成指针树会破坏 ECS locality；强行取消层次又难以表达骨架、挂点和编辑器组织。因此实际引擎常维护独立 Transform graph，同时 entity/component 存其他 runtime 数据，再生成 render scene。

核心抓手

- 层次关系与组件存储是正交问题
- Transform update 需要拓扑/脏标记
- ECS 与 Render Scene 也常分离

工程 Gotcha

把 Scene Graph 当成 ECS 的旧版本。

高频追问树

- 父节点移动如何高效更新子树？
- 如何避免每帧递归遍历整棵场景树？

面试官在听什么

是否能把不同数据结构按职责组合。

Q73 • 游戏引擎 Serialization 系统如何设计?

难度: • 公开面经/官方资料可核验: [M1]

30 秒回答

序列化需要把对象状态映射到稳定格式, 并处理类型元数据、对象引用、版本迁移、endianness、可选字段与资产 GUID。最难的是 schema evolution。

深入拆解

成熟系统不应把裸内存 dump 作为通用格式, 因为 compiler padding、指针、版本和平台都会改变。字段应有稳定 ID/名字, 引用保存 GUID/handle 而不是地址。加载时可以先构造对象表, 再第二阶段 resolve references。版本升级可通过 per-type migration 或 tagged field 兼容。编辑器格式可以可读, runtime cooked 格式则追求解析速度和紧凑。

核心抓手

- 不要序列化裸指针
- 稳定字段 ID/版本
- 两阶段 resolve reference
- 编辑器格式与 runtime cooked format 可不同

工程 Gotcha

把 C++ struct 直接 fwrite 到磁盘。

高频追问树

- 字段重命名如何兼容?
- 循环对象图如何序列化?
- 如何支持增量保存和 diff?

面试官在听什么

是否理解数据长期演化, 而不仅是“读写 JSON”。

Q74 • Asset Pipeline 为什么不能直接运行时读取 FBX/PSD?

难度： • 高频能力重构

30 秒回答

源资产格式为创作工具服务，解析重、信息冗余且跨版本复杂；引擎通常离线 import/cook 成面向 runtime 的紧凑格式，提前完成压缩、LOD、shader/material 编译和依赖分析。

深入拆解

Cook 阶段可以把 expensive work 从帧时移到构建时，例如 mesh optimize、texture transcode、animation compression、shader permutation。runtime 文件应支持快速随机/流式读取和版本校验。更进一步会按目标平台 cook：PC 用 BC，移动端用 ASTC，主机可能有专用容器。Asset pipeline 是引擎性能的一半，因为“最便宜的 runtime 工作是根本不在 runtime 做”。

核心抓手

- source format != runtime format
- 离线预计算换运行时性能
- 按平台 cook
- 依赖图用于增量构建与打包

工程 Gotcha

运行时临时解析复杂 DCC 格式，造成启动和 streaming hitch。

高频追问树

- 如何做增量 cook?
- shader cache 与 asset cook 如何关联?
- 如何保证源资产改动只重建依赖部分?

面试官在听什么

是否理解 offline/runtime 分工。

Q75 · 如何设计 Asset GUID?

难度： · 高频能力重构

30 秒回答

资产 identity 应与文件路径解耦。用稳定 GUID 作为引用键，再由数据库映射到当前路径/包位置，移动或重命名文件不会破坏引用。

深入拆解

还要解决 GUID 的生成、复制、冲突和版本控制。若复制一个资产文件时也复制元数据 GUID，会造成 collision；编辑器需要检测并重生成。runtime 可以把长 GUID cook 成紧凑 integer handle。依赖图、redirector、软引用和异步加载都建立在稳定 identity 上。

核心抓手

- 路径是 location，不是 identity
- GUID 需要冲突检测
- cook 后可映射成紧凑索引
- 软引用允许目标暂未加载

工程 Gotcha

直接把绝对路径写入场景文件。跨机器、重命名、打包都会出问题。

高频追问树

- 资源删除后旧 GUID 怎么处理？
- redirector 是什么？
- 如何实现软引用与弱依赖？

面试官在听什么

是否具备数据血缘/长期维护意识。

Q76 · 什么是 Resource Streaming?

难度： · 公开面经/官方资料可核验：[O4]

30 秒回答

在有限 RAM/VRAM 和 I/O 带宽下，根据相机、重要度和未来需求按需加载/卸载纹理 mip、mesh LOD、音频块等，而不是一次把整个世界放进内存。

深入拆解

一个 streaming system 需要需求估计、priority、I/O scheduling、decode/transcode、GPU upload、residency budget 和 eviction。最危险的是“刚看到才加载”：SSD、解压、upload 都有延迟，因此需要根据相机速度和预测提前请求。纹理 virtual/streaming mip、geometry streaming 都是把大资源切成可独立 residency 的 page/chunk。

核心抓手

- 需求预测
- 预算与驱逐
- 异步 I/O + decode + upload pipeline
- 粒度必须可独立加载

工程 Gotcha

只讨论“异步加载线程”，忽略 GPU residency 和预算。

高频追问树

- 如何避免 streaming thrashing?
- 相机瞬移怎么办?
- IO priority 如何与 gameplay 关键资源结合?

面试官在听什么

是否能设计端到端数据流。

Q77 • Game Thread、Render Thread、RHI Thread 如何交互？

难度： · 公开面经/官方资料可核验：[O2], [O3]

30 秒回答

Game Thread 生成世界状态，Render Thread 构建平台无关渲染命令，RHI Thread/后端把命令翻译提交到 DX12/Vulkan/Metal 等 API；线程间通过命令队列、proxy 和 fence 管理状态与生命周期。

深入拆解

Unreal 官方 Parallel Rendering 文档明确描述 Render Thread 作为 frontend，把平台无关 command list 交给 RHI backend，在支持的平台并行翻译。这个架构的价值是把 gameplay state、render state 和 API submission 拆开，避免锁住 Game Thread。资源销毁、动态更新和 frame lag 都必须有确定的 ownership protocol。

核心抓手

- Game state 不应被 RHI 直接读取
- Render Thread 负责 render-domain data
- RHI 是跨平台 backend interface
- 命令/资源生命周期跨帧异步

工程 Gotcha

把多线程理解成“按顺序三个函数调用”，忽略它们同时在处理不同帧。

高频追问树

- RHI Thread 一定存在吗？
- 资源创建在哪个线程？
- 如何设计 deferred deletion queue？

面试官在听什么

是否理解现代引擎多线程 renderer 的分层。

Q78 · 什么是 Render Graph?

难度： · 公开面经/官方资料可核验：[O3]

30 秒回答

把一帧渲染表示成 Pass 与 Resource 的依赖图。每个 pass 声明读写，编译阶段自动确定执行顺序、resource lifetime、barrier、transient allocation/aliasing。

深入拆解

传统手写 render pass 容易出现资源生命周期过长、barrier 过保守、隐藏依赖。Render Graph 把“我要读什么/写什么”声明化，系统可做 dead-pass culling、自动 transition、别名复用。真正难点是 side effect、external resource/import、跨 queue async compute 和 debugging；如果 pass 隐式访问全局资源，图就无法正确优化。

核心抓手

- 声明式 resource dependency
- 自动 lifetime/barrier
- transient memory aliasing
- 能做 pass culling 和 async scheduling

工程 Gotcha

把 Render Graph 当成“画流程图”的工具，不理解它是资源/同步编译器。

高频追问树

- 如何处理跨 frame history texture?
- Async Compute 如何进入 graph?
- 为什么隐藏 global state 会破坏 graph?

面试官在听什么

是否理解现代 renderer 的资源调度核心。

Q79 · 为什么 Shader Variant 会爆炸？如何控制？

难度： · 高频能力重构

30 秒回答

多个独立 feature keyword 形成笛卡尔积，variant 数量指数增长，导致编译时间、磁盘、PSO cache 和运行时 warmup 都爆炸。

深入拆解

控制方法包括离线 pruning 无效组合、把低分支成本 feature 改为 runtime branch、用 specialization constants、按 material/shading model 分层、统计真实使用组合并构建 shader library。现代显式 API 还会出现 PSO permutation: shader + blend + depth + raster state 的组合进一步放大问题。

核心抓手

- 组合爆炸来自独立维度相乘
- prune 不可能组合
- runtime branch 与 static variant 是 trade-off
- PSO cache 是相关问题

工程 Gotcha

为每个 checkbox 都加一个 compile-time define。

高频追问树

- 动态分支一定慢吗？
- 如何做 shader warmup？
- PSO cache 如何跨驱动版本管理？

面试官在听什么

是否理解 build-time 与 runtime 的系统性成本。

Q80 · 如何设计跨平台 RHI?

难度： · 公开面经/官方资料可核验：[M1], [O3]

30 秒回答

对 Buffer、Texture、Pipeline、CommandList、Fence、Descriptor 等核心概念提供平台无关接口，下层分别映射 DX12/Vulkan/Metal；同时必须保留足够能力，不要为了“统一”抹平现代 API 特性。

深入拆解

好的 RHI 往往以 capability/query 驱动差异，而不是大量 if(platform)。需要决定同步模型、resource state、descriptor binding、queue、memory allocation 由上层还是后端负责。如果抽象过高，无法使用 Vulkan/DX12 的显式并行；过低则 renderer 到处暴露 API 差异。实际引擎通常还在 RHI 上再有更高层 Render Graph。

核心抓手

- 抽象稳定概念而非 API 函数名
- capability-driven
- 显式 API 的同步/descriptor 不能完全隐藏
- RHI 与 Render Graph 分层

工程 Gotcha

设计成“把 OpenGL 函数重命名成统一接口”，无法表达现代 API。

高频追问树

- Metal 没有完全对应的概念怎么办？
- RHI 是否暴露 resource state？
- 如何支持平台专有扩展？

面试官在听什么

是否具备跨 API 抽象能力。

第 9 章

动画、物理、AI 与网络

本章目标

理解实时模拟子系统如何在固定帧预算、确定性和网络延迟之间做工程权衡。

必须掌握： skinning; IK; fixed timestep; broad/narrow phase; GJK; NavMesh; prediction/reconciliation

Q81 • Skeletal Animation 如何工作?

难度: • 高频能力重构

30 秒回答

每个顶点保存若干 bone index 与 weight; 当前骨骼矩阵乘 inverse bind pose 得 skinning matrix, 顶点最终位置是多个骨骼变换后的加权和。

深入拆解

Bind pose 定义 mesh 与 skeleton 的基准关系。对 bone i, 常见 skin matrix = currentGlobal_i * inverseBind_i, 再对 position/normal 做 weighted blend。CPU skinning 简单但带宽重, GPU vertex/compute skinning 更常见。Compute skinning 可把结果复用给多个 pass, 但需要额外 buffer 与同步; vertex skinning 每 pass 重算但不存中间结果。

核心抓手

- inverse bind pose 把顶点带回骨骼局部参考
- 权重通常和为 1
- CPU/GPU/compute skinning 有不同带宽权衡
- 法线变换也要处理

工程 Gotcha

只写 $\sum w_i M_i v$, 却不知道 M_i 为什么需要 inverse bind。

高频追问树

- 为什么常限制 4/8 bones per vertex?
- Compute Skinning 什么时候更划算?
- 如何做 skinning bounds?

面试官在听什么

是否理解动画数据到渲染数据的完整链。

Q82 · 为什么 Linear Blend Skinning 会出现 Candy Wrapper?

难度： · 高频能力重构

30 秒回答

LBS 直接线性混合变换矩阵/变换后位置，旋转空间本身不是线性空间，尤其扭转时会造成截面收缩、体积丢失。

深入拆解

两个相反旋转的矩阵线性加权后不再是纯旋转，会包含缩放/剪切。Dual Quaternion Skinning 用 dual quaternion 对刚体变换做更合理插值，可显著改善 twist，但对非均匀 scale 支持更复杂。工业角色还会使用 corrective blendshape、pose-space deformation 修复特定关节。

核心抓手

- 线性混合旋转会产生非刚性变换
- DQS 改善 twist/体积
- corrective shape 解决艺术特定形变

工程 Gotcha

认为权重归一化就能解决 candy wrapper。

高频追问树

- DQS 为什么对非均匀 scale 麻烦？
- elbow collapse 与 twist artifact 有什么不同？

面试官在听什么

是否能把数学缺陷与视觉结果联系起来。

Q83 · FK 与 IK 有什么区别？

难度： · 高频能力重构

30 秒回答

FK 从根到末端给定关节旋转，直接得到末端姿态；IK 给定末端目标，反求一组关节参数。

深入拆解

两骨骼手臂/腿可用解析法快速求解；复杂链常用 CCD、FABRIK、Jacobian 等迭代算法。实际动画系统通常“基础动画/FK + IK 修正”，例如脚贴地、手抓武器。IK 还要处理 joint limits、pole vector、不可达目标和 temporal stability。

核心抓手

- FK 是 forward transform propagation
- IK 是约束求解
- 解析法快但适用结构有限
- 动画常混合 FK 与 IK

工程 Gotcha

把 IK 说成“让动画更自然”，没有说明求解目标和约束。

高频追问树

- FABRIK 和 CCD 如何比较？
- 两骨 IK 的 pole vector 有什么作用？
- 不可达目标怎么处理？

面试官在听什么

是否理解动画系统是约束求解问题。

Q84 • Animation State Machine 与 Blend Tree 分别解决什么？

难度： • 高频能力重构

30 秒回答

State Machine 管理离散动作状态与转移；Blend Tree 在速度、方向等连续参数空间内混合多个 animation clip。

深入拆解

状态机解决“何时从 Idle 进入 Jump”，Blend Tree 解决“0~6m/s 的 locomotion 应在 walk/run 之间如何连续过渡”。复杂系统还会使用 layered blend、additive animation、sync group 和 motion matching。过大的状态机会变成“意大利面”，因此现代系统逐渐把 locomotion 数据驱动化。

核心抓手

- 离散状态 vs 连续插值
- transition condition 与 blend duration
- layer/additive 用于身体局部叠加

工程 Gotcha

把所有动作都堆成状态机，导致组合状态指数膨胀。

高频追问树

- 如何同步不同步幅的 walk/run？
- 什么是 additive animation？
- Motion Matching 为什么减少手工状态？

面试官在听什么

是否理解动画控制层的可扩展性问题。

Q85 • Physics Fixed Timestep 为什么重要?

难度: • 高频能力重构

30 秒回答

固定 Δt 让数值积分、碰撞和约束求解的稳定性更可控, 也更利于可复现模拟; 渲染帧率变化时用 accumulator 跑 0/1/多次 physics step。

深入拆解

经典 loop: accumulator += frameDt; while accumulator >= fixedDt 执行 physics; 渲染时可在前后两个物理状态间 interpolation。若单帧卡顿导致 accumulator 巨大, 可能出现 “spiral of death”, 因此应限制最大 catch-up steps。固定步长也不保证完全 deterministic: 浮点、线程调度、平台指令仍会造成差异。

核心抓手

- 固定步长提高稳定性
- accumulator loop
- 渲染可插值
- 限制 catch-up 防止 spiral of death

工程 Gotcha

认为 fixed timestep 自动意味着跨机器 bitwise deterministic。

高频追问树

- 为什么可变 dt 会让弹簧/约束不稳定?
- 网络 lockstep 对 physics 有什么额外要求?

面试官在听什么

是否把数值模拟放在 real-time loop 中考虑。

Q86 • Collision Detection 为什么分 Broad Phase 和 Narrow Phase?

难度： • 高频能力重构

30 秒回答

N 个对象两两精确测试是 $O(N^2)$ 且单次成本高; Broad Phase 用 AABB/BVH/SAP/Spatial Hash 快速找潜在 pair, Narrow Phase 再做精确 shape test。

深入拆解

Broad Phase 的目标是高召回: 不能漏掉真实碰撞, 但允许 false positive。动态刚体常用 Dynamic AABB Tree、Sweep-and-Prune; 大量粒子可用 uniform grid。Narrow Phase 再用 SAT、GJK/EPA、convex-convex 或 mesh contact。最终还要进入 contact manifold 与 constraint solver。

核心抓手

- Broad phase 只生成候选
- 允许 false positive 不允许漏碰撞
- 不同运动分布适合不同结构
- collision pipeline 不止“相交判断”

工程 Gotcha

在 broad phase 就使用复杂 OBB/mesh 精确求交, 失去分层意义。

高频追问树

- SAP 为什么适合时间连续的动态物体?
- fat AABB 有什么作用?
- CCD 放在哪一层?

面试官在听什么

是否理解物理引擎的分层性能设计。

Q87 · GJK 在解决什么问题？

难度： · 高频能力重构

30 秒回答

GJK 判断两个凸体是否相交：A 与 B 相交等价于 Minkowski Difference A B 包含原点；算法用 support mapping 迭代构造 simplex，逼近/包围原点。

深入拆解

优势是不需要显式构造 Minkowski Difference，只要每个 convex shape 能给定方向 d 的最远点 $\text{support}(d)$ 。在 3D simplex 从点/线/三角形到四面体。若需要穿透深度与接触法线，可在相交后用 EPA。GJK 的难点是数值鲁棒性、终止条件和退化 simplex。

核心抓手

- Minkowski Difference
- support function
- simplex 迭代
- EPA 补充 penetration depth

工程 Gotcha

把 GJK 说成“一个 SAT 的优化版”，两者思路不同。

高频追问树

- 为什么只需要 support mapping?
- GJK 如何算距离而不只是相交?
- EPA 如何继续扩展 polytope?

面试官在听什么

是否理解凸几何算法的核心变换。

Q88 • NavMesh 为什么比规则 Grid 更适合 3D 游戏角色?

难度: • 高频能力重构

30 秒回答

NavMesh 用凸多边形覆盖可行走连续区域, 节点更少、路径不受栅格方向限制; A* 在 polygon adjacency 上搜索, 再用 funnel algorithm 穿过 portal 得到直线路径。

深入拆解

NavMesh 还天然编码角色可通行区域, 可在 bake 时考虑 slope、step height、clearance。缺点是动态世界更新复杂、不同角色半径可能需要多套数据或 runtime query。开放世界会把 navmesh 分 tile 流式加载, 动态 obstacle 用局部 carve 或 avoidance, 而不一定重烘全局。

核心抓手

- polygon graph 降节点数量
- funnel 做路径拉直
- bake 可加入角色几何约束
- tile 化适合 streaming

工程 Gotcha

认为 NavMesh 直接存“最终路径”, 实际上它是可通行空间表示。

高频追问树

- Funnel algorithm 的 portal 是什么?
- 动态障碍怎么处理?
- 不同体型 NPC 共用一张 NavMesh 怎么办?

面试官在听什么

是否理解导航从表示到查询的完整系统。

Q89 • Client Prediction 为什么需要 Server Reconciliation?

难度： · 高频能力重构

30 秒回答

客户端为了手感立即模拟本地输入，但服务器保持权威状态；服务器状态返回后，客户端若发现偏差，需要回滚到确认状态并重放尚未确认的输入。

深入拆解

客户端为每个 input 带 sequence number, 保存预测历史。Server ack 某序号并返回 authoritative state；客户端丢弃已确认输入，把状态校正后 replay remaining inputs。视觉上常用 smoothing 避免瞬移。预测必须与服务器规则足够一致，否则会频繁纠正。作弊防护也要求客户端不能直接声明最终位置。

核心抓手

- 低延迟与权威性并存
- 输入序号/历史缓存
- rewind + replay
- 视觉平滑与逻辑状态分离

工程 Gotcha

只做“收到服务器位置就 lerp”，没有解决未来输入的状态连续性。

高频追问树

- 丢包/乱序怎么处理？
- projectile 是否适合 client prediction？
- lag compensation 与 reconciliation 区别？

面试官在听什么

是否理解实时网络的时序问题。

Q90 • Lockstep 和 State Replication 如何选择?

难度: • 高频能力重构

30 秒回答

Lockstep 主要同步输入,各端做相同确定性模拟,带宽低但要求强 determinism; State Replication 由服务器模拟并同步状态,更通用但带宽更高。

深入拆解

RTS 常能容忍输入延迟并拥有大量单位,因此 lockstep 很合适; FPS/动作游戏需要服务器权威和局部预测,更偏 snapshot/state replication。State replication 还会做 delta compression、interest management、snapshot interpolation。Lockstep 若任一客户端慢会拖累整体,并对浮点/随机数/执行顺序高度敏感。

核心抓手

- Lockstep 省状态带宽但要求确定性
- Replication 适合服务器权威
- interest management 是扩展关键
- 选择取决于玩法与延迟模型

工程 Gotcha

把“P2P=Lockstep, Client-Server=Replication”当绝对绑定。

高频追问树

- Rollback netcode 属于哪一类思路?
- 如何做 snapshot interpolation?
- 确定性随机数怎么管理?

面试官在听什么

是否从游戏类型和网络约束选择同步模型。

第 10 章

GPU API、现代引擎与性能优化

本章目标

现代引擎高阶题集中在同步、GPU-driven、Bindless、Render Graph、Nanite/Lumen 与 profiling。

必须掌握： Vulkan sync； barrier scopes； Bindless； GPU-driven； Compute； profiling； mobile GPU； Nanite/Lumen/Reverse-Z； 60FPS 系统设计

Q91 • Vulkan 中 Fence、Semaphore、Barrier 分别解决什么？

难度： · 公开面经/官方资料可核验：[O6]

30 秒回答

Fence 主要让 CPU 观察 GPU completion；Semaphore 用于 GPU submission/queue 之间同步；Pipeline/Memory Barrier 在命令流中建立执行与内存依赖，并常伴随 image layout transition。

深入拆解

不能只背三句话。Fence 常绑定 frame context, CPU 等它后才能安全重用 per-frame command/resource。Semaphore, 尤其 timeline semaphore, 可表示队列间生产-消费顺序。Barrier 则指定 src/dst stage 与 access scope: 谁的哪些写必须在谁的哪些读之前可见。Vulkan Synchronization2 把 stage/access 信息更清晰地组合在 barrier 结构中。

核心抓手

- Fence=CPU/GPU completion observation
- Semaphore=GPU work dependency
- Barrier=execution+memory dependency
- layout transition 也是同步语义的一部分

工程 Gotcha

认为 semaphore 会自动让 CPU 等待，或 barrier 等同于“GPU 全部停住”。

高频追问树

- Binary vs Timeline Semaphore?
- 为什么 event 很少作为第一选择?
- Queue family ownership transfer 怎么做?

面试官在听什么

是否真正掌握显式 API 的同步模型。

Q92 · 为什么 Pipeline Barrier 写得太保守会掉性能？

难度： · 公开面经/官方资料可核验：[O6], [O7]

30 秒回答

过宽的 source/destination stage 和 access scope 会让本来可重叠的 GPU 工作等待不相关任务完成，形成 synchronization bubble。

深入拆解

例如把所有依赖写成 ALL_COMMANDS→ALL_COMMANDS 等于告诉驱动“几乎什么都等什么”，失去 fine-grained pipeline overlap。正确做法是只同步真正产生数据的 src stage/access 与真正消费数据的 dst stage/access。性能工具可在 GPU timeline 看到空洞。Vulkan 官方 profiling 资料也把 overly restrictive barriers 列为常见 bubble 根因。

核心抓手

- barrier scope 越精确越有并行机会
- 同步要同时考虑 execution 与 memory
- GPU timeline 能定位 bubble
- 正确性优先，再最小化范围

工程 Gotcha

为了性能删掉 barrier，出现偶现 corruption。优化顺序必须先证明依赖正确。

高频追问树

- 为什么 TOP_OF_PIPE/BOTTOM_OF_PIPE 常不是正确“通用答案”？
- Async Compute 如何受 barrier 影响？

面试官在听什么

是否能在 correctness 与并行度之间精确推理。

Q93 · 什么是 Bindless Rendering?

难度: · 公开面经/官方资料可核验: [M3]

30 秒回答

把大量 texture/buffer descriptor 放进大表, 材质只存资源索引, shader 按索引访问, 减少每 draw 重新绑定资源并支持大量异构材质与 GPU-driven submission。

深入拆解

Bindless 的难点从“bind 调用”转移到 descriptor heap 管理、稳定 index、资源生命周期、更新安全和 residency。删除纹理时不能立刻复用 index, 因为旧 frame command 可能仍引用它; 通常需要 deferred free/generation handle。不同 API 有 descriptor indexing/argument buffer 等不同实现, RHI 应抽象能力而非假设完全相同。

核心抓手

- 资源表 + index
- 减少 CPU binding churn
- 适合 GPU-driven/material diversity
- descriptor lifetime 是核心难点

工程 Gotcha

认为 Bindless 意味着“资源无限”。硬件 descriptor 数、VRAM 和 residency 仍有限。

高频追问树

- descriptor index 何时可以复用?
- non-uniform indexing 有什么注意?
- Bindless 与 texture atlas 如何比较?

面试官在听什么

是否理解 Bindless 的系统级资源管理成本。

Q94 • GPU-Driven Rendering 是什么？

难度： • 高频能力重构

30 秒回答

把可见性、LOD、实例 compact、甚至 draw command generation 从 CPU 移到 GPU，通过 compute + indirect draw 减少 CPU submission，并让海量对象按 GPU 可见性结果直接绘制。

深入拆解

典型流程：GPU 读取 instance/meshlet data → frustum/occlusion culling → prefix-sum/atomic compact visible list → 写 indirect args → draw indirect。Hi-Z 负责 occlusion，Bindless/mesh shader 解决材质与几何异构。难点是 GPU culling 自身成本、全局原子竞争、indirect buffer 同步和调试可见性。不是对象越少越适合 GPU-driven。

核心抓手

- compute culling
- visible list compaction
- indirect draw
- 常与 Bindless/Hi-Z/Meshlet 配合

工程 Gotcha

把所有 CPU logic 都搬到 GPU。GPU 适合高并行规则工作，不适合复杂串行 gameplay。

高频追问树

- 如何避免 atomic bottleneck?
- 上一帧 Hi-Z culling 的误差如何处理?
- Mesh Shader 如何改变 command generation?

面试官在听什么

是否理解现代大规模 renderer 的数据流。

Q95 • Compute Shader 可以在游戏引擎里做什么？

难度： · 公开面经/官方资料可核验：[M1]

30 秒回答

适合大量独立或局部协作的数据并行任务：粒子、culling、skinning、lighting、prefix sum、image processing、simulation、mip 生成等。

深入拆解

选择 Compute 不应只因为“GPU 并行”。要考虑 dispatch 规模、memory access coalescing、shared memory、workgroup occupancy、barrier、读写带宽以及与 graphics queue 的同步。对很小任务，dispatch/sync overhead 可能得不偿失；对 irregular pointer-heavy 算法，CPU 可能更适合。好的回答会拿一个具体算法说明线程映射与数据布局。

核心抓手

- 数据并行度
- memory bandwidth/occupancy
- workgroup shared memory
- dispatch 与 graphics 同步成本

工程 Gotcha

列十个用途但说不出一个 compute kernel 如何组织线程。

高频追问树

- prefix sum 为什么是 GPU 基础算法？
- shared memory 什么时候有用？
- Async Compute 什么情况下能并行？

面试官在听什么

是否理解 GPU 计算模型而不是 API 名称。

Q96 · 一帧只有 20 FPS，你如何定位问题？

难度： · 公开面经/官方资料可核验：[M2], [O7]

30 秒回答

20 FPS 50ms/frame。第一步不是“优化代码”，而是确定 CPU-bound、GPU-bound、sync-bound、I/O hitch 还是内存/温度问题，再用 profiler 找 critical path。

深入拆解

CPU 先看主线程/worker timeline、sampling profile、任务等待；GPU 看 pass timestamp、pipeline counters、overdraw、bandwidth、occupancy；再检查 CPU-GPU queue 是否一方 starving。优化必须闭环：measure → hypothesis → change → measure。平均 FPS 会掩盖 hitch，应同时看 frame-time percentile (P95/P99) 和 spike。移动端还要考虑 thermal throttling。

核心抓手

- 先分类瓶颈
- 看 frame time 而非只看 FPS
- CPU/GPU timeline 找 critical path
- P99 hitch 与平均性能都重要

工程 Gotcha

一看到 GPU 慢就降分辨率；可能真正是 CPU submission 或同步 bubble。

高频追问树

- 如何判断 CPU-bound？
- 如何做 A/B 实验证明优化有效？
- 为什么 GPU profiler 自身可能改变 timing？

面试官在听什么

这是工程岗最重要的“问题定位能力”题。

Q97 · 为什么移动 GPU 和桌面 GPU 优化思路不同？

难度： · 公开面经/官方资料可核验：[M2], [M6]

30 秒回答

移动芯片大量采用 tile-based 架构，片上 tile memory 能显著减少外部内存带宽；因此 render pass 合并、attachment load/store、overdraw、带宽与能耗比桌面更敏感。

深入拆解

TBDR/tiler 会先分 bin，再在 tile 内完成部分 raster/shading，尽量不把中间 framebuffer 往 DRAM 反复读写。错误的 pass 边界或必须 store/reload 的 attachment 会破坏优势。移动端还受统一内存、功耗和 thermal budget 约束，峰值性能不能长时间持续。桌面优化经验不能直接照搬。

核心抓手

- tile memory 降低带宽
- render pass/load-store 很关键
- 功耗/温度是长期预算
- overdraw 和透明常很昂贵

工程 Gotcha

认为“移动 GPU 就是更慢的桌面 GPU”。

高频追问树

- 为什么 mobile deferred 仍然可行？
- MSAA 在 tile GPU 上为什么可能更友好？
- discard/alpha test 对 tiler 有何影响？

面试官在听什么

是否具备平台架构敏感性。

Q98 • Nanite 到底解决了什么问题？

难度： · 公开面经/官方资料可核验：[O4]

30 秒回答

Nanite 是虚拟化几何系统：用内部压缩/层次表示、细粒度 streaming 与可见细节选择，使 renderer 只为屏幕真正可见的像素级细节付费，降低传统手工 LOD 与巨量 draw/triangle 管理压力。

深入拆解

Epic 官方将其定义为 virtualized geometry system，目标包括 pixel-scale detail、高 object count、压缩数据和 fine-grained streaming。面试时不应把它简化成“自动 LOD”。真正的问题是如何让几何 detail representation、cluster/meshlet culling、streaming 和 rasterization 协同，使源资产三角形数量不再直接等价于每帧工作量。仍有材质、透明、变形和平台方面的约束。

核心抓手

- virtualized geometry
- hierarchical detail/culling
- fine-grained streaming
- 工作量更接近可见细节而非源多边形数

工程 Gotcha

说“Nanite 可以无限三角形”。任何系统都有 GPU、带宽、内存和特性限制。

高频追问树

- 为什么传统 discrete LOD 不够？
- Nanite 与 Virtual Shadow Map 有什么关系？
- geometry streaming 的 page 如何管理？

面试官在听什么

是否能从系统目标而非营销词解释现代技术。

Q99 • Lumen 的设计目标是什么？

难度： · 公开面经/官方资料可核验：[O5]

30 秒回答

Lumen 面向全动态 GI 与反射，在大规模场景中用多种场景表示、追踪与缓存近似间接光，而不是依赖预烘 lightmap。

深入拆解

Epic 官方描述 Lumen 可提供动态 diffuse interreflection 与 indirect specular, 并与 Nanite、World Partition 等系统配合。实时 GI 的核心矛盾是高维光传输与有限帧预算，因此实现通常会混合 screen-space 信息、software/hardware ray tracing、低分辨率 indirect lighting、radiance cache 和 temporal reconstruction。高质量回答应讨论“哪些信息可复用、哪里会漏光/缺屏幕外信息、如何控制更新预算”。

核心抓手

- 动态 GI + reflections
- 多种 tracing path/caching
- 低分辨率计算 + temporal reuse
- 与大世界/高细节 geometry 协同

工程 Gotcha

把 Lumen 说成“实时 Path Tracing”。它是多种近似技术组合，不是简单全路径追踪。

高频追问树

- Screen-space tracing 的信息缺失怎么补？
- radiance cache 为什么有用？
- 动态场景如何控制 cache 更新？

面试官在听什么

是否能从 GI 约束推导混合系统设计。

Q100 · 系统设计：如何设计一个稳定 60 FPS 的开放世界引擎？

难度： · 公开面经/官方资料可核验：[01], [02], [03], [04], [05], [07]

30 秒回答

先把 16.67ms/frame 当硬预算，再按 CPU、GPU、RAM/VRAM、I/O 建预算与 telemetry；世界分区、资源 streaming、可见性、任务系统、Render Graph 和异步上传必须围绕“只处理玩家当前需要的数据”协同。

深入拆解

一个完整方案至少包含：World Partition/streaming cells；纹理/几何/动画分块加载；frustum+LOD+occlusion+GPU culling；Game/Job/Render/RHI 多线程；Render Graph 管理 transient resource/barrier；GPU graphics/compute/copy overlap；arena/pool 与 deferred GPU resource deletion；帧时 profiler 和 P99 hitch tracking。关键是说明每个系统如何影响同一个 frame budget，而不是罗列技术名。设计时还要预留平台差异和降级路径。

核心抓手

- 预算驱动而不是 feature 驱动
- 只处理可见/需要的数据
- CPU/GPU/I/O 流水并行
- 所有优化必须有 profiler 反馈闭环

工程 Gotcha

堆砌 Nanite、Lumen、ECS、Job System 等名词，却没有资源/线程依赖和帧预算。

高频追问树

- 相机高速移动时 streaming 如何保证？
- CPU 8ms、GPU 20ms 先优化谁？
- 如何定义 memory budget 与 eviction？
- 如何设计画质 scalability？

面试官在听什么

这是把前 99 题串成“工程系统能力”的终极题。

第 A 章

30 题冲刺清单

如果面试只剩 3~5 天，建议优先把下列 30 题练到“脱稿讲 3 分钟 + 接两层追问”：

题号	冲刺主题
Q01	虚函数 / vtable / devirtualization
Q04	shared_ptr / weak_ptr / ownership
Q07	vector 扩容与失效规则
Q10	Reflection / Serialization / Codegen
Q13	Memory Pool
Q14	Frame Arena
Q15	AoS / SoA
Q16	ECS
Q19	Cache Miss
Q23	Job System
Q26	Lock-free SPSC
Q27	Acquire / Release
Q29	Game Thread / Render Thread
Q32	A*
Q39	Octree / BVH
Q41	坐标空间
Q43	Normal Matrix
Q50	Perspective-correct Interpolation
Q51	Raster Pipeline
Q54	Early-Z
Q56	Hi-Z
Q57	Forward / Deferred
Q61	Rendering Equation
Q62	Cook-Torrance
Q65	Shadow Mapping
Q69	Importance Sampling / MIS
Q70	BVH for Ray Tracing
Q78	Render Graph
Q91	Vulkan Synchronization
Q96	Profiling: 20 FPS 如何定位

第 B 章

项目追问模板

面试官真正喜欢沿项目代码追问。建议给自己的每个项目准备下面 8 个问题：

1. 这个模块的性能目标是什么？用什么指标证明？
2. 最初方案是什么，为什么不够？
3. 数据结构为何这样选？替代方案是什么？
4. CPU/GPU memory layout 如何设计？
5. 哪些地方存在同步或生命周期风险？
6. 最难复现的 bug 是什么？你如何定位？
7. 如果规模扩大 10 倍，第一处瓶颈在哪里？
8. 如果换成移动端/主机/另一图形 API，哪些假设会失效？

第 C 章

资料来源与进一步阅读

公开面经说明：以下 M 类资料均为候选人个人回忆，用于判断“考察主题是否真实出现过”，不视为公司官方题库。题目在本书中均经过重新表述和扩展。

- [M1] 腾讯游戏引擎研发实习面经（候选人回忆）
<https://www.nowcoder.com/discuss/353157907686563840>
- [M2] 腾讯 IEG 游戏引擎面经（2025，候选人回忆）
<https://www.nowcoder.com/feed/main/detail/8a93fe3305fd491ba7c8705d233c2a80>
- [M3] Unity 中国引擎研发实习生面经（候选人回忆）
<https://www.nowcoder.com/feed/main/detail/94e7409152494799b0bab3ac069b812c?sourceSSR=dynamic>
- [M4] 腾讯游戏引擎实习面经：PBR / Path Tracing / Reverse-Z / Early-Z（候选人回忆）
<https://www.nowcoder.com/discuss/567436318603476992>
- [M5] 腾讯光子游戏客户端（引擎方向）面经（候选人回忆）
<https://www.nowcoder.com/discuss/613002354509570048>
- [M6] 莉莉丝游戏引擎面经：Early-Z / PCF / PCSS / GBuffer（候选人回忆）
<https://www.nowcoder.com/feed/main/detail/af85d0328d014297bb3d1ae86e40612a>
- [O1] Epic Games - Programming Career Paths
<https://www.epicgames.com/site/earlycareers/career-paths?lang=en>
- [O2] Unreal Engine 5.8 - Threaded Rendering
<https://dev.epicgames.com/documentation/unreal-engine/threaded-rendering-in-unreal-engine?lang=en-US>
- [O3] Unreal Engine 5.8 - Parallel Rendering Overview
<https://dev.epicgames.com/documentation/unreal-engine/parallel-rendering-overview?lang=en-US>
- [O4] Unreal Engine 5.8 - Nanite Virtualized Geometry
<https://dev.epicgames.com/documentation/unreal-engine/nanite-in-unreal-engine?lang=en-US>
- [O5] Unreal Engine 5.8 - Lumen Global Illumination and Reflections
<https://dev.epicgames.com/documentation/en-us/unreal-engine/lumen-global-illumination?lang=en-US>
- [O6] Vulkan Documentation - Synchronization2

https://docs.vulkan.org/guide/latest/extensions/VK_KHR_synchronization2.html

[O7] Vulkan Documentation - Profiling

<https://docs.vulkan.org/guide/latest/profiling.html>

推荐基础资料

- Real-Time Rendering（实时渲染体系与工程视角）
- Physically Based Rendering（渲染方程、Monte Carlo、BSDF 与 Path Tracing）
- Game Engine Architecture（引擎系统分层与工程实践）
- C++ Concurrency in Action（C++ 内存模型与并发）
- Vulkan / Direct3D 12 官方文档与 GPU 厂商 profiling 工具文档
- Unreal Engine 官方 Rendering / RHI / Nanite / Lumen / RDG 文档

结语：从“知道”到“能定位”

游戏引擎岗位最难的地方不是知识面宽，而是每个知识点都必须继续向下落到真实系统：

概念 → 原理 → 数据结构 → CPU/GPU 执行 → Trade-off → Profiling

如果能对任何一项技术回答“为什么这样设计、什么条件下失败、如何用工具证明”，你已经从背题进入了真正的 Engine Programmer 思维。